

```
import javax.swing.*;
import javax.swing.table.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.table.DefaultTableModel;
import java.time.LocalDateTime;
import java.util.List;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;
import java.sql.*;
import java.util.function.Function;
public class Main {

    public static JFrame mainFrame;
    public static CardLayout cardLayout;
    public static JPanel mainPanel;
    public static Inventory inventory;
    public static Menu menu;
    public static Restaurant restaurant; // NEW: Restaurant instance
    public static ArrayList<RestaurantTable> tables = new
    ArrayList<>();
    public static ArrayList<Reservation> reservations = new
    ArrayList<>();
    public static int reservationId = 1;
    public static ArrayList<Order> orders = new ArrayList<>();
    public static ArrayList<Bill> bills = new ArrayList<>();
    public static ArrayList<Payment> payments = new ArrayList<>();
```

```

public static void main(String[] args) throws
ClassNotFoundException, SQLException {
    DatabaseIntegration.initializeIngredientsTable();
    // 1. Load the JDBC driver
    Class.forName("com.mysql.cj.jdbc.Driver");
    //  Ensure default menu items exist in database
    try (Connection con = DriverManager.getConnection(

```

```

"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC", "root",
"Angela@3root");

```

```

    Statement st = con.createStatement() {
    st.executeUpdate(
        "CREATE TABLE IF NOT EXISTS menu_items (" +
        " item_id INT PRIMARY KEY, " +
        " name VARCHAR(100), " +
        " category VARCHAR(100), " +
        " total_amount decimal(10,2) , " +
        " is_available BOOLEAN, " +
        " ingredients TEXT)"
    );

```

```

    st.executeUpdate(
        "INSERT IGNORE INTO menu_items (item_id,
name, category, total_amount, is_available, ingredients) VALUES
" +
        "(1, 'Margherita Pizza', 'Main Course', 250, TRUE,
['\"Cheese\", \"Tomato\"])," +
        "(2, 'Red Sauce Pasta', 'Main Course', 350, TRUE,
['\"Penne\", \"Tomato Sauce\"])," +

```

```

        "(3, 'Veg Burger', 'Snack', 150, TRUE,
        ['\"Bun\\\", \"Patty\\\", \"Lettuce\\\"])," +
        "(4, 'Coke', 'Beverage', 50, TRUE, ['\"Carbonated
        Water\\\"]');"
    );
    System.out.println("✅ Default menu items ensured in
    database.");
    } catch (SQLException e) {
        System.err.println("❌ Error inserting default menu
        items: " + e.getMessage());
    }
}

```

//ADD DATA

```

inventory = new Inventory();
inventory.ingredients.put("Tomato", new Ingredient("Tomato", 2.0,
"kg", 5.0 ));
inventory.ingredients.put("Cheese", new Ingredient("Cheese",
1.0, "kg", 2.0));
inventory.ingredients.put("Onion", new Ingredient("Onion", 1.0,
"kg", 3.5));
inventory.ingredients.put("Flour", new Ingredient("Flour", 5.0,
"kg", 10.0));
menu = new Menu();
MenuItem pizza = new MenuItem(1, "Margherita Pizza", "Main
Course", 250.0, List.of("Cheese", "Tomato"));
menu.addItem(pizza);
MenuItem pasta = new MenuItem(2, "Red Sauce Pasta", "Main
Course", 350.0, List.of("Cheese", "Tomato"));

```

```
menu.addItem(pasta);
System.out.println("Search result: " +
menu.searchByName("Pizza"));
// NEW: Initialize Restaurant with sample employees
restaurant = new Restaurant("Spice Garden");
restaurant.addEmployee(new Chef("Rajesh Kumar"));
restaurant.addEmployee(new Chef("Priya Sharma"));
restaurant.addEmployee(new Waiter("Amit Patel"));
restaurant.addEmployee(new Waiter("Sneha Reddy"));
restaurant.addEmployee(new Manager("Vikram Singh"));
// Initialize tables
// Initialize database tables
try {
RestaurantTableDAO.initializeTables();
} catch (SQLException e) {
System.err.println("Error initializing tables: " + e.getMessage());
}
// Load tables from database
try {
tables = RestaurantTableDAO.loadTables();
System.out.println("✅ Loaded " + tables.size() + " tables from
database");
} catch (SQLException e) {
System.err.println("❌ Error loading tables: " + e.getMessage());
// Fallback to in-memory tables
int tableId = 1;
for (int i = 0; i < 8; i++) tables.add(new
RestaurantTable(tableId++, 4));
for (int i = 0; i < 6; i++) tables.add(new
RestaurantTable(tableId++, 6));
```

```

for (int i = 0; i < 2; i++) tables.add(new
RestaurantTable(tableId++, 10));
for (int i = 0; i < 6; i++) tables.add(new
RestaurantTable(tableId++, 2));
}
// Load existing reservations from database
try {
reservations = ReservationDAO.loadReservations();
if (!reservations.isEmpty()) {
reservationId = reservations.get(reservations.size() -
1).reservationId + 1;
}
System.out.println("✅ Loaded " + reservations.size() + "
reservations from database");
} catch (SQLException e) {
System.err.println("❌ Error loading reservations: " +
e.getMessage());
}
// Initialize main frame once
mainFrame = new JFrame("Spice Garden");
mainFrame.setSize(800, 600);
mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE
);
mainFrame.setLocationRelativeTo(null);
// Set up CardLayout
cardLayout = new CardLayout();
mainPanel = new JPanel(cardLayout);
mainFrame.add(mainPanel);
// Add all panels
mainPanel.add(createWelcomePanel(), "WELCOME");

```

```

mainPanel.add(createRoleSelectionPanel(),
"ROLE_SELECTION");
mainPanel.add(createManagerPanel(), "MANAGER");
mainPanel.add(createDisplayMenuPanel(), "MENU_DISPLAY");
mainPanel.add(createRemoveItemPanel(), "REMOVE_ITEM");
mainPanel.add(createAddItemPanel(), "ADD_ITEM");
mainPanel.add(createInventoryPanel(), "INVENTORY");
mainPanel.add(createReservationPanel(), "RESERVATION");
mainPanel.add(createChefPanel(), "CHEF");
mainPanel.add(createWaiterPanel(), "WAITER");
mainPanel.add(createOrderManagementPanel(),
"ORDER_MANAGEMENT");
mainPanel.add(createEmployeeManagementPanel(),
"EMPLOYEE_MANAGEMENT"); // NEW
// Show welcome panel first
cardLayout.show(mainPanel, "WELCOME");
mainFrame.setVisible(true);
}

private static JPanel createWelcomePanel() {
JPanel panel = new JPanel();
panel.setBackground(new Color(255, 228, 196));
panel.setLayout(new BorderLayout());
JLabel welcomeLabel = new JLabel("~Welcome to Spice
Garden~", SwingConstants.CENTER);
welcomeLabel.setFont(new Font("Serif", Font.BOLD, 24));
welcomeLabel.setForeground(new Color(139, 69, 19));
panel.add(welcomeLabel, BorderLayout.NORTH);
JButton enterButton = new JButton("Enter");
enterButton.setFont(new Font("SansSerif", Font.PLAIN, 18));
enterButton.setBackground(new Color(255, 160, 122));

```

```

enterButton.setForeground(Color.WHITE);
enterButton.setFocusPainted(false);
enterButton.setBorder(BorderFactory.createEmptyBorder(10, 20,
10, 20));
enterButton.addActionListener(e -> {
    cardLayout.show(mainPanel, "ROLE_SELECTION");
});
panel.add(enterButton, BorderLayout.CENTER);
return panel;
}

private static JPanel createRoleSelectionPanel() {
JPanel panel = new JPanel();
panel.setBackground(new Color(255, 248, 220));
panel.setLayout(new BorderLayout());
JLabel label = new JLabel("Select your role:");
label.setFont(new Font("Serif", Font.BOLD, 40));
label.setHorizontalAlignment(SwingConstants.CENTER);
panel.add(label, BorderLayout.NORTH);
JPanel buttonPanel = new JPanel();
buttonPanel.setBackground(new Color(255, 248, 220));
buttonPanel.setLayout(new GridLayout(2, 2, 10, 10));
String[] roles = {"Manager", "Chef", "Waiter", "Receptionist"};
for (String role : roles) {
    JButton roleButton = new JButton(role);
    roleButton.setFont(new Font("SansSerif", Font.PLAIN, 16));
    roleButton.setBackground(new Color(255, 160, 122));
    roleButton.setForeground(Color.WHITE);
    roleButton.setFocusPainted(false);
    roleButton.addActionListener(e -> {

```

```

JOptionPane.showMessageDialog(mainFrame, "You
selected: " + role);
    if(role.equals("Manager")) {
        cardLayout.show(mainPanel, "MANAGER");
    } else if(role.equals("Receptionist")) {
        cardLayout.show(mainPanel, "RESERVATION");
    } else if(role.equals("Chef")) {
        cardLayout.show(mainPanel, "CHEF");
    } else if(role.equals("Waiter")) {
        cardLayout.show(mainPanel, "WAITER");
    }
});
    buttonPanel.add(roleButton);
}
panel.add(buttonPanel, BorderLayout.CENTER);
JButton backButton = new JButton("Back");
backButton.addActionListener(e -> cardLayout.show(mainPanel,
"WELCOME"));
panel.add(backButton, BorderLayout.SOUTH);
return panel;
}

private static JPanel createManagerPanel() {
    JPanel panel = new JPanel();
    panel.setBackground(new Color(244, 164, 96));
    panel.setLayout(new BorderLayout());

    JLabel label = new JLabel("Select an action:");
    label.setFont(new Font("Serif", Font.BOLD, 40));
    label.setHorizontalAlignment(SwingConstants.CENTER);
    panel.add(label, BorderLayout.NORTH);

```



```
JPanel buttonPanel = new JPanel();  
buttonPanel.setBackground(new Color(244, 164, 96));  
buttonPanel.setLayout(new GridLayout(5, 2, 10, 10)); //
```

Changed to 5 rows

```
String[] actions = {"DisplayMenu", "RemoveItem",  
"AddItem", "View Ingredients",  
"Order Management", "View Reports",  
"Employee Management", "View Past Employees",  
"View Daily Revenue", ""};
```

```
for(String action : actions) {  
    if(action.isEmpty()) {  
        buttonPanel.add(new JLabel()); // Empty space  
        continue;  
    }  
    JButton actionButton = new JButton(action);  
    actionButton.setBackground(new Color(245, 222, 179));  
    actionButton.setForeground(new Color(34, 139, 34));  
    actionButton.setBorderPainted(false);  
    actionButton.addActionListener(e -> {  
        if(action.equals("DisplayMenu")) {  
            cardLayout.show(mainPanel, "MENU_DISPLAY");  
        } else if(action.equals("RemoveItem")) {  
            cardLayout.show(mainPanel, "REMOVE_ITEM");  
        } else if(action.equals("AddItem")) {  
            cardLayout.show(mainPanel, "ADD_ITEM");  
        } else if(action.equals("View Ingredients")) {  
            cardLayout.show(mainPanel, "INVENTORY");  
        }  
    });  
}
```

```

        } else if(action.equals("Order Management")) {
            cardLayout.show(mainPanel,
"ORDER_MANAGEMENT");
        } else if(action.equals("View Reports")) {
            viewReports();
        } else if(action.equals("Employee Management")) {
            cardLayout.show(mainPanel,
"EMPLOYEE_MANAGEMENT");
        } else if(action.equals("View Past Employees")) {
            viewPastEmployees();
        } else if(action.equals("View Daily Revenue")) {
            viewDailyRevenue();
        }
    });
    buttonPanel.add(actionButton);
}

```

```

panel.add(buttonPanel, BorderLayout.CENTER);

```

```

JButton backButton = new JButton("Back");
backButton.setBackground(new Color(245, 222, 179));
backButton.setForeground(new Color(34, 139, 34));
backButton.setBorderPainted(false);
backButton.addActionListener(e ->
cardLayout.show(mainPanel, "ROLE_SELECTION"));
panel.add(backButton, BorderLayout.SOUTH);

return panel;
}

```

```
// ===== NEW: EMPLOYEE MANAGEMENT  
PANEL =====
```

```
private static JPanel createEmployeeManagementPanel() {  
    JPanel panel = new JPanel(new BorderLayout(10, 10));  
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10,  
    10));
```

```
    panel.setBackground(new Color(245, 245, 245));
```

```
// Header
```

```
JLabel header = new JLabel("🏠 " + restaurant.name + " —  
Employee Management", SwingConstants.CENTER);
```

```
header.setFont(new Font("Segoe UI", Font.BOLD, 24));
```

```
header.setForeground(Color.WHITE);
```

```
JPanel headerPanel = new JPanel();
```

```
headerPanel.setBackground(new Color(44, 62, 80));
```

```
headerPanel.add(header);
```

```
panel.add(headerPanel, BorderLayout.NORTH);
```

```
// Table
```

```
String[] columns = {"ID", "Name", "Role", "Action"};
```

```
DefaultTableModel employeeModel = new
```

```
DefaultTableModel(columns, 0);
```

```
JTable employeeTable = new JTable(employeeModel);
```

```
employeeTable.setRowHeight(28);
```

```
employeeTable.setFont(new Font("SansSerif", Font.PLAIN, 14));
```

```
employeeTable.getTableHeader().setFont(new Font("SansSerif",  
Font.BOLD, 15));
```

```
employeeTable.getTableHeader().setBackground(new Color(52,  
152, 219));
```

```
employeeTable.getTableHeader().setForeground(Color.WHITE);
```

```
JScrollPane scrollPane = new JScrollPane(employeeTable);
```

```
panel.add(scrollPane, BorderLayout.CENTER);
```

// Method to refresh table

```
Runnable updateTable = () -> {  
    employeeModel.setRowCount(0);  
    for (Employee e : restaurant.getEmployees()) {  
        employeeModel.addRow(new Object[]{  
            e.getId(), e.getName(), e.getRole(), e.getAction()  
        });  
    }  
};
```

updateTable.run();

// Button Panel

```
JPanel buttonPanel = new JPanel(new GridLayout(2, 4, 10, 10));  
JButton addChefBtn = createStyledButton("Add Chef", new  
Color(231, 76, 60));  
JButton addWaiterBtn = createStyledButton("Add Waiter", new  
Color(46, 204, 113));  
JButton addManagerBtn = createStyledButton("Add Manager",  
new Color(155, 89, 182));  
JButton viewBtn = createStyledButton("Refresh", new Color(52,  
152, 219));  
JButton deleteBtn = createStyledButton("Remove Employee",  
new Color(241, 196, 15));  
JButton demoBtn = createStyledButton("Demo Workflow", new  
Color(230, 126, 34));  
JButton backBtn = createStyledButton("Back", new Color(44, 62,  
80));
```

// Button Actions

```
addChefBtn.addActionListener(e -> {  
    String name = JOptionPane.showInputDialog(mainFrame,  
"Enter Chef name:");
```

```

    if (name != null && !name.trim().isEmpty()) {
        restaurant.addEmployee(new Chef(name));
        JOptionPane.showMessageDialog(mainFrame, "Chef " +
name + " added successfully!");
        updateTable.run();
    }
});
addWaiterBtn.addActionListener(e -> {
    String name = JOptionPane.showInputDialog(mainFrame,
"Enter Waiter name:");
    if (name != null && !name.trim().isEmpty()) {
        restaurant.addEmployee(new Waiter(name));
        JOptionPane.showMessageDialog(mainFrame, "Waiter " +
name + " added successfully!");
        updateTable.run();
    }
});
addManagerBtn.addActionListener(e -> {
    String name = JOptionPane.showInputDialog(mainFrame,
"Enter Manager name:");
    if (name != null && !name.trim().isEmpty()) {
        restaurant.addEmployee(new Manager(name));
        JOptionPane.showMessageDialog(mainFrame, "Manager " +
name + " added successfully!");
        updateTable.run();
    }
});
viewBtn.addActionListener(e -> updateTable.run());
deleteBtn.addActionListener(e -> {
    try {

```

```

        String idStr = JOptionPane.showInputDialog(mainFrame,
"Enter Employee ID to remove:");
        if (idStr == null) return;
        int id = Integer.parseInt(idStr);
        Employee emp = restaurant.findEmployee(id);
        if (emp == null) {
            JOptionPane.showMessageDialog(mainFrame, "❌
Employee not found!");
            return;
        }
        // Add to past employees before removing
        restaurant.addPastEmployee(emp);
        restaurant.getEmployees().remove(emp);
        JOptionPane.showMessageDialog(mainFrame, "🗑️
Employee " + emp.getName() + " moved to past employees.");
        updateTable.run();
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(mainFrame, "⚠️ Invalid
ID!");
    }
});

demoBtn.addActionListener(e -> {
    String message =
        "🧩 Restaurant Workflow Simulation\n\n" +
        "1 Waiter takes an order from customer.\n" +
        "2 Chef prepares the order.\n" +
        "3 Waiter serves the meal.\n" +
        "4 Manager reviews operations and handles payment.\n\n" +
        "💡 Integrated Employee Management System";
    JTextArea area = new JTextArea(message);

```

```

        area.setFont(new Font("Monospaced", Font.PLAIN, 14));
        area.setEditable(false);
        area.setBackground(new Color(250, 250, 250));
        JOptionPane.showMessageDialog(mainFrame, new
        JScrollPane(area),
            "Restaurant Workflow",
        JOptionPane.INFORMATION_MESSAGE);
    });
    backButton.addActionListener(e -> cardLayout.show(mainPanel,
    "MANAGER"));
    buttonPanel.add(addChefBtn);
    buttonPanel.add(addWaiterBtn);
    buttonPanel.add(addManagerBtn);
    buttonPanel.add(viewBtn);
    buttonPanel.add(deleteBtn);
    buttonPanel.add(demoBtn);
    buttonPanel.add(backBtn);
    buttonPanel.add(new JLabel()); // Empty space
    panel.add(buttonPanel, BorderLayout.SOUTH);
    return panel;
}

private static JButton createStyledButton(String text, Color color)
{
    JButton b = new JButton(text);
    b.setFont(new Font("Segoe UI", Font.BOLD, 13));
    b.setBackground(color);
    b.setForeground(Color.WHITE);
    b.setFocusPainted(false);
    return b;
}

```

```

private static JPanel createDisplayMenuPanel() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBackground(new Color(255, 248, 220));
    JLabel title = new JLabel("Available Menu Items");
    title.setFont(new Font("Serif", Font.BOLD, 32));
    title.setHorizontalAlignment(SwingConstants.CENTER);
    panel.add(title, BorderLayout.NORTH);
    String[] columns = {"Item ID", "Name", "Category", "Total
Amount (₹)", "Available", "Ingredients"};
    DefaultTableModel tableModel = new
DefaultTableModel(columns, 0);
    JTable table = new JTable(tableModel);
    JScrollPane scrollPane = new JScrollPane(table);
    panel.add(scrollPane, BorderLayout.CENTER);
    JPanel bottomPanel = new JPanel();
    bottomPanel.setBackground(new Color(255, 248, 220));
    JButton refreshButton = new JButton("Refresh");
    JButton backButton = new JButton("Back");
    // ✓ Refresh Button — reloads menu items
    refreshButton.addActionListener(e -> {
        tableModel.setRowCount(0);
        try (Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC",
        "root", "Angela@3root");
        PreparedStatement ps = con.prepareStatement("SELECT
* FROM menu_items");
        ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {

```



```

    int id = rs.getInt("item_id");
    String name = rs.getString("name");
    String category = rs.getString("category");
    double totalAmount = rs.getDouble("total_amount");
    boolean available = rs.getBoolean("is_available");
    String ingredients = rs.getString("ingredients");
    tableModel.addRow(new Object[]{
        id, name, category, "₹ " + String.format("%.2f",
totalAmount),
        available ? "Yes  " : "No  ", ingredients
    });
}
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(panel,
        "Error loading menu items: " + ex.getMessage(),
        "Database Error", JOptionPane.ERROR_MESSAGE);
}
});
//  FIX: Back Button — navigate back to Manager main page
backButton.addActionListener(e -> {
    cardLayout.show(mainPanel, "MANAGER"); //  Switch
back to manager dashboard
});
bottomPanel.add(refreshButton);
bottomPanel.add(backButton);
panel.add(bottomPanel, BorderLayout.SOUTH);
//  Auto-load menu after panel is fully added
SwingUtilities.invokeLater(() -> refreshButton.doClick());
return panel;
}

```

```

private static JPanel createRemoveItemPanel() {
    JPanel panel = new JPanel(new BorderLayout());
    panel.setBorder(BorderFactory.createEmptyBorder(10,
10, 10, 10));
    String[] columns = {"ID", "Name", "Category", "Total
Amount", "Available", "Ingredients"};
    DefaultTableModel model = new
DefaultTableModel(columns, 0) {
        @Override
        public boolean isCellEditable(int row, int column) {
            return false; // make table read-only
        }
    };
    JTable table = new JTable(model);

table.setSelectionMode(ListSelectionModel.SINGLE_SELECTIO
N);

    JScrollPane scrollPane = new JScrollPane(table);
    panel.add(scrollPane, BorderLayout.CENTER);
    // Helper: parse JSON array string like ["a","b"] into
List<String>
    Function<String, List<String>> parseJsonArray = json -> {
        List<String> list = new ArrayList<>();
        if (json == null) return list;
        String s = json.trim();
        if (s.startsWith("[") && s.endsWith("]")) {
            s = s.substring(1, s.length() - 1).trim();
            if (!s.isEmpty()) {
                // split on commas that are not inside quotes
                (simple approach for simple arrays)
            }
        }
    };
}

```

```

        // remove wrapping quotes and unescape \" if
present
        String[] parts =
s.split("(?<=\\\"|\\s*,\\s*(?=\\\"))|(?<=\\w)\\s*,\\s*(?=\\\"|\\s*,\\s*)");
        for (String p : parts) {
            p = p.trim();
            if (p.startsWith("\\\"") && p.endsWith("\\\"") &&
p.length() >= 2) {
                p = p.substring(1, p.length() - 1).replace("\\\\\"",
"\\");
            }
            if (!p.isEmpty()) list.add(p);
        }
    }
}
return list;
};

```

// Method to refresh the table data from DB and also
update in-memory menu list

```

Runnable refreshTable = () -> {
    model.setRowCount(0); // Clear existing rows
    // Clear in-memory menu items first so we re-sync with
DB
    menu.getItems().clear();
    String url =
"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC";
    String user = "root";
    String password = "Angela@3root";

```

```

        String sql = "SELECT item_id, name, category,
total_amount, is_available, ingredients FROM menu_items
ORDER BY item_id";
        try (Connection con =
DriverManager.getConnection(url, user, password);
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                int id = rs.getInt("item_id");
                String name = rs.getString("name");
                String category = rs.getString("category");
                double total_amount=
rs.getDouble("total_amount");
                boolean available = rs.getBoolean("is_available");
                String ingredientsJson =
rs.getString("ingredients");
                List<String> ingredients =
parseJSONArray.apply(ingredientsJson);
                // Add row to table (show ingredients JSON or
readable joined string)
                String ingredientsDisplay = ingredients.isEmpty() ?
"" : String.join(", ", ingredients);
                model.addRow(new Object[]{id, name, category,
total_amount, available ? "Yes" : "No", ingredientsDisplay});
                // Create MenuItem and add to in-memory menu
(note: MenuItem constructor sets available = true currently)
                // If you want to track DB availability in-memory
you may want to add a setter in MenuItem.
                MenuItem mi = new MenuItem(id, name, category,
total_amount, ingredients);

```

// if you need to reflect DB availability in-memory
and MenuItem lacks a setter, you can toggle if necessary:

```
if (!available && mi.isAvailable()) {  
    mi.toggleAvailability(); // sets to false  
} else if (available && !mi.isAvailable()) {  
    mi.toggleAvailability(); // ensure true  
}
```

`menu.addItem(mi);` // this will also attempt to insert
into DB if your `Menu.addItem` inserts — to avoid double-insert,
use direct list access:

// NOTE: If `Menu.addItem` writes to DB, avoid
calling it here. Instead add directly to the internal list:

```
// For safety, add directly:  
// menu.getItems().add(mi);  
// To avoid double-insert, remove the
```

`menu.addItem(mi)` above and use the direct add below:

```
}
```

// --- IMPORTANT: avoid double DB inserts caused
by `Menu.addItem` writing to DB.

// To ensure safe behavior, rebuild in-memory list
without re-inserting:

// (Replace the above loop's `menu.addItem(mi)` with
this approach.)

```
} catch (SQLException ex) {  
    JOptionPane.showMessageDialog(panel, "Error  
loading menu items: " + ex.getMessage(), "Database Error",  
JOptionPane.ERROR_MESSAGE);  
}  
};
```

// IMPORTANT: The Menu.addItem(...) method in your code performs a DB insert. When refreshing from DB we must NOT call that.

// So replace the refreshTable runnable above with this safer implementation (to be sure, we override here):

// Declare as final

```
final Runnable refreshTable1 = () -> {  
    model.setRowCount(0); // Clear existing rows  
    try (Connection con = DriverManager.getConnection(  
        "jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP  
        ublicKeyRetrieval=true&serverTimezone=UTC", "root",  
        "Angela@3root");  
        Statement st = con.createStatement();  
        ResultSet rs = st.executeQuery("SELECT * FROM  
menu_items")) {  
        while (rs.next()) {  
            Object[] row = {  
                rs.getInt("item_id"),  
                rs.getString("name"),  
                rs.getString("category"),  
                rs.getDouble("total_amount"),  
                rs.getInt("is_available") == 1 ? "Yes" : "No"  
            };  
            model.addRow(row);  
        }  
    } catch (SQLException ex) {  
        JOptionPane.showMessageDialog(mainFrame,  
            "Error loading menu: " + ex.getMessage(),
```

```

        "Database Error",
JOptionPane.ERROR_MESSAGE);
    }
};
// Initial load
refreshTable1.run();
JButton removeBtn = new JButton("Remove Item");
removeBtn.addActionListener(e -> {
    int selectedRow = table.getSelectedRow();
    if (selectedRow >= 0) {
        int itemId =
Integer.parseInt(table.getValueAt(selectedRow, 0).toString());
        int confirm =
JOptionPane.showConfirmDialog(mainFrame,
        "Are you sure you want to remove this item?",
        "Confirm Removal",
JOptionPane.YES_NO_OPTION);
        if (confirm == JOptionPane.YES_OPTION) {
            String url =
"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC";
            String user = "root";
            String password = "Angela@3root";
            String sql = "DELETE FROM menu_items WHERE
item_id = ?";
            try (Connection con =
DriverManager.getConnection(url, user, password);
                PreparedStatement ps =
con.prepareStatement(sql)) {
                ps.setInt(1, itemId);

```

```

        int rows = ps.executeUpdate();
        if (rows > 0) {
            // remove from in-memory list safely
            menu.getItems().removeIf(i -> i.getId() ==
itemId);

            // refresh the UI from DB
            refreshTable1.run();

JOptionPane.showMessageDialog(mainFrame, "Item removed
successfully!");
        } else {

JOptionPane.showMessageDialog(mainFrame, "No item found
with ID: " + itemId);
        }
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(mainFrame,
"Database error: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }
} else {
    JOptionPane.showMessageDialog(mainFrame,
"Please select an item first.");
}
});
JButton refreshBtn = new JButton("Refresh");
refreshBtn.addActionListener(e -> refreshTable1.run());
JButton backBtn = new JButton("Back");

```



```

        backBtn.addActionListener(e ->
cardLayout.show(mainPanel, "MANAGER"));
        JPanel buttonPanel = new JPanel();
        buttonPanel.add(removeBtn);
        buttonPanel.add(refreshBtn);
        buttonPanel.add(backBtn);
        panel.add(buttonPanel, BorderLayout.SOUTH);
        return panel;
    }

    private static JPanel createAddItemPanel() {
        JPanel panel = new JPanel();
        panel.setLayout(new BorderLayout());
        // Create form panel
        JPanel formPanel = new JPanel(new GridLayout(5, 2, 10, 10));
        formPanel.setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));
        JTextField idField = new JTextField();
        JTextField nameField = new JTextField();
        JTextField categoryField = new JTextField();
        JTextField total_amountField = new JTextField();
        JTextField ingredientsField = new JTextField();
        formPanel.add(new JLabel("ID:"));
        formPanel.add(idField);
        formPanel.add(new JLabel("Name:"));
        formPanel.add(nameField);
        formPanel.add(new JLabel("Category:"));
        formPanel.add(categoryField);
        formPanel.add(new JLabel("Total Amount (₹):"));
        formPanel.add(total_amountField);
        formPanel.add(new JLabel("Ingredients (comma-separated):"));
    }

```

```

formPanel.add(ingredientsField);
panel.add(formPanel, BorderLayout.CENTER);
// Buttons panel
JPanel buttonPanel = new JPanel();
JButton addButton = new JButton("Add Item");
addButton.addActionListener(e -> {
    try {
        int id = Integer.parseInt(idField.getText().trim());
        String name = nameField.getText().trim();
        String category = categoryField.getText().trim();
        if (name.isEmpty() || category.isEmpty()) {
            throw new Exception("Name and Category cannot be
empty.");
        }
        if (name.matches("\\d+") || category.matches("\\d+")) {
            throw new Exception("Name and Category cannot be
numbers.");
        }
        double total_amount =
Double.parseDouble(total_amountField.getText().trim());
        if (total_amount < 10.0 || total_amount > 1000.0) {
            throw new Exception("total_amount must be between
10.0 and 1000.0");
        }
        //  Convert ingredient input into ArrayList
        List<String> ingredients = new ArrayList<>();
        String ingText = ingredientsField.getText().trim();
        if (!ingText.isEmpty()) {
            ingredients = new
ArrayList<>(Arrays.asList(ingText.split(", ")));

```

```

    for (int i = 0; i < ingredients.size(); i++) {
        ingredients.set(i, ingredients.get(i).trim());
    }
}

// ✓ Safely convert ArrayList to valid JSON
StringBuilder jsonBuilder = new StringBuilder("[");
for (int i = 0; i < ingredients.size(); i++) {
    jsonBuilder.append("\"")
        .append(ingredients.get(i).replace("\"", "\\\""))
        .append("\"");
    if (i < ingredients.size() - 1) jsonBuilder.append(",");
}
jsonBuilder.append("]");
String jsonIngredients = jsonBuilder.toString();
// ✓ JDBC connection
Class.forName("com.mysql.cj.jdbc.Driver");
try (Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC", "root",
    "Angela@3root")) {
    // Check for duplicate ID
    String checkQuery = "SELECT COUNT(*) FROM
menu_items WHERE item_id = ?";
    PreparedStatement checkPs =
conn.prepareStatement(checkQuery);
    checkPs.setInt(1, id);
    ResultSet rs = checkPs.executeQuery();
    rs.next();
    if (rs.getInt(1) > 0) {

```

```

        throw new Exception("Item ID already exists. Please
use a unique ID.");
    }
    //  Insert data
    String sql = "INSERT INTO menu_items (item_id, name,
category, total_amount, is_available, ingredients) VALUES (?, ?,
?, ?, ?, ?)";
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setInt(1, id);
    ps.setString(2, name);
    ps.setString(3, category);
    ps.setDouble(4, total_amount);
    ps.setBoolean(5, true); // default available
    ps.setString(6, jsonIngredients);
    ps.executeUpdate();
}
JOptionPane.showMessageDialog(mainFrame, "Item added
successfully to database!");
// Reset fields
idField.setText("");
nameField.setText("");
categoryField.setText("");
total_amountField.setText("");
ingredientsField.setText("");
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(mainFrame, "ID and Total
Amount must be numeric.");
} catch (SQLIntegrityConstraintViolationException ex) {
    JOptionPane.showMessageDialog(mainFrame, "Duplicate ID
or Name. Please use unique values.");
}

```

```

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(mainFrame,
ex.getMessage());
    }
});
JButton backButton = new JButton("Back");
backButton.addActionListener(e -> cardLayout.show(mainPanel,
"MANAGER"));
buttonPanel.add(addButton);
buttonPanel.add(backButton);
panel.add(buttonPanel, BorderLayout.SOUTH);
return panel;
}

private static JPanel createInventoryPanel() {
InventoryUI inventoryUI = new InventoryUI(inventory);
return inventoryUI.getContentPane();
}

private static JPanel createReservationPanel() {
JPanel panel = new JPanel(new BorderLayout(10, 10));
panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10,
10));
panel.setBackground(new Color(240, 248, 255));
// Table View
String[] columns = {"Table ID", "Capacity", "Status"};
DefaultTableModel tableModel = new
DefaultTableModel(columns, 0);
JTable tableView = new JTable(tableModel);
JScrollPane tableScrollPane = new JScrollPane(tableView);
// Method to refresh table view
Runnable refreshTableView = () -> {

```

```

    tableModel.setRowCount(0);
    for (RestaurantTable t : tables) {
        tableModel.addRow(new Object[]{t.table_id, t.capacity,
t.getStatus()});
    }
};
refreshTableView.run();
panel.add(tableScrollPane, BorderLayout.CENTER);
// Buttons
JPanel buttonPanel = new JPanel(new GridLayout(2, 3, 10, 10));
JButton btnReserve = new JButton("Make Reservation");
JButton btnWalkIn = new JButton("Walk-in Allocation");
JButton btnViewRes = new JButton("View Reservations");
JButton btnFree = new JButton("Free a Table");
JButton btnRefresh = new JButton("Refresh");
JButton btnBack = new JButton("Back");
buttonPanel.add(btnReserve);
buttonPanel.add(btnWalkIn);
buttonPanel.add(btnViewRes);
buttonPanel.add(btnFree);
buttonPanel.add(btnRefresh);
buttonPanel.add(btnBack);
panel.add(buttonPanel, BorderLayout.SOUTH);
// Reservation Display
JTextArea reservationArea = new JTextArea(15, 25);
reservationArea.setEditable(false);
reservationArea.setBorder(BorderFactory.createTitledBorder("Res
ervation Details"));
panel.add(new JScrollPane(reservationArea),
BorderLayout.EAST);

```

// Button Actions

```
btnReserve.addActionListener(e -> makeReservation(true,
refreshTableView));
btnWalkIn.addActionListener(e -> makeReservation(false,
refreshTableView));
btnViewRes.addActionListener(e ->
viewReservations(reservationArea));
btnFree.addActionListener(e -> {
    freeTable(refreshTableView);
});
btnRefresh.addActionListener(e -> refreshTableView.run());
btnBack.addActionListener(e -> cardLayout.show(mainPanel,
"ROLE_SELECTION"));
return panel;
}

private static void makeReservation(boolean isReservation,
Runnable refreshTableView) {
    try {
        String dt = isReservation ?
JOptionPane.showInputDialog(mainFrame, "Enter Date & Time:")
: "Walk-in";
        if (dt == null || (isReservation && dt.trim().isEmpty()))
return;

        String cidStr =
JOptionPane.showInputDialog(mainFrame, "Enter Customer
ID:");
        if (cidStr == null) return;
        if (!cidStr.matches("\\d+")) {
```

```
        JOptionPane.showMessageDialog(mainFrame,
"Customer ID must be a number!");
        return;
    }
    int cid = Integer.parseInt(cidStr);
    if (cid <= 0) {
        JOptionPane.showMessageDialog(mainFrame,
"Customer ID must be a positive number!");
        return;
    }
}
```

```
String name =
JOptionPane.showInputDialog(mainFrame, "Enter Customer
Name:");
    if (name == null || name.trim().isEmpty()) return;
    if (name.matches("\\d+")) {
        JOptionPane.showMessageDialog(mainFrame,
"Name cannot be all numbers!");
        return;
    }
}
```

```
String phone =
JOptionPane.showInputDialog(mainFrame, "Enter Phone (10
digits):");
    if (phone == null) return;
    phone = phone.trim();
    if (!phone.matches("\\d{10}")) {
        JOptionPane.showMessageDialog(mainFrame,
"Phone number must be exactly 10 digits!");
        return;
    }
}
```



```

    }

    String email =
OptionPane.showInputDialog(mainFrame, "Enter Email:");
    if (email == null || email.trim().isEmpty()) return;

    String seatsStr =
OptionPane.showInputDialog(mainFrame, "Seats Needed:");
    if (seatsStr == null) return;
    if (!seatsStr.matches("\\d+")) {
        JOptionPane.showMessageDialog(mainFrame,
"Seats must be a positive number!");
        return;
    }
    int seatsNeeded = Integer.parseInt(seatsStr);
    if (seatsNeeded <= 0) {
        JOptionPane.showMessageDialog(mainFrame,
"Seats must be greater than zero!");
        return;
    }

    // Reload tables from database
    try {
        tables = RestaurantTableDAO.loadTables();
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(mainFrame,
"Error loading table status: " + e.getMessage());
        return;
    }

```

```

    int totalAvailable = tables.stream()
        .filter(t -> t.isAvailable)
        .mapToInt(t -> t.capacity)
        .sum();

    if (seatsNeeded > totalAvailable) {
        JOptionPane.showMessageDialog(mainFrame, "Not
enough seats available! Available: " + totalAvailable);
        return;
    }

    ArrayList<RestaurantTable> availableTables = new
ArrayList<>();
    for (RestaurantTable t : tables) {
        if (t.isAvailable) availableTables.add(t);
    }

    ArrayList<RestaurantTable> best =
getBestTableCombination(availableTables, seatsNeeded);
    if (best == null || best.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame,
"Could not allocate tables!");
        return;
    }

    Customer c = new Customer(cid, name, phone, email);
    Reservation reservation = new
Reservation(reservationId, c, best, dt);

    // Save to database

```

```

    try {
        int dbReservationId =
ReservationDAO.saveReservation(reservation);
        reservation.reservationId = dbReservationId;
        reservations.add(reservation);
        reservationId = dbReservationId + 1;

        for (RestaurantTable t : best) {
            t.isAvailable = false;
        }

        JOptionPane.showMessageDialog(mainFrame,
            "✓ Reservation confirmed!\nReservation ID: " +
dbReservationId);

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(mainFrame,
            "✗ Error saving reservation: " + e.getMessage());
        e.printStackTrace();
        return;
    }

    refreshTableView.run();

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(mainFrame, "Invalid
number format!");
    } catch (Exception e) {
        JOptionPane.showMessageDialog(mainFrame, "An
error occurred: " + e.getMessage());
    }

```

```

        e.printStackTrace();
    }
}

private static ArrayList<RestaurantTable>
getBestTableCombination(ArrayList<RestaurantTable>
availableTables,
    int seatsNeeded) {
    ArrayList<RestaurantTable> best = new ArrayList<>();
    // Pick the smallest single table that can fit all guests
    RestaurantTable chosen = null;
    int minCapacity = Integer.MAX_VALUE;
    for (RestaurantTable t : availableTables) {
        if (t.capacity >= seatsNeeded && t.capacity < minCapacity) {
            chosen = t;
            minCapacity = t.capacity;
        }
    }
    if (chosen != null) {
        best.add(chosen);
    }
    return best; // ✅ Only one table, never null
}

private static void findCombination(ArrayList<RestaurantTable>
tables, int index, int seatsNeeded,
    ArrayList<RestaurantTable> current,
    ArrayList<RestaurantTable> best) {
    int sum = current.stream().mapToInt(t -> t.capacity).sum();
    if (sum >= seatsNeeded) {
        if (best.isEmpty() || sum < best.stream().mapToInt(t ->
t.capacity).sum()) {

```

```

best.clear();
best.addAll(new ArrayList<>(current));
}
return;
}
if (index >= tables.size()) return;
// Choose current table
current.add(tables.get(index));
findCombination(tables, index + 1, seatsNeeded, current, best);
// Backtrack (don't choose current table)
current.remove(current.size() - 1);
findCombination(tables, index + 1, seatsNeeded, current, best);
}

private static void viewReservations(JTextArea
reservationArea) {
    try {
        reservations = ReservationDAO.loadReservations();
    } catch (SQLException e) {
        reservationArea.setText("Error loading reservations: " +
e.getMessage());
        return;
    }

    if (reservations.isEmpty()) {
        reservationArea.setText("No active reservations
found!");
        return;
    }

    StringBuilder sb = new StringBuilder();

```

```

        for (Reservation r : reservations) {
            sb.append("\n--- Reservation ID: ")
            sb.append(r.reservationId).append(" ---\n")
            sb.append("Customer: ").append(r.customer.name)
            sb.append(" ")
            sb.append(r.customer.phone).append("\n")
            sb.append("Tables: ");
            int totalSeats = 0;
            for (RestaurantTable t : r.allocatedTables) {
                sb.append(t.table_id).append("(").append(t.capacity).append(" ");
                totalSeats += t.capacity;
            }
            sb.append("\nTotal Seats: ").append(totalSeats)
                .append("\nDate & Time: ").append(r.dateTime)
                .append("\n-----\n");
        }
        reservationArea.setText(sb.toString());
    }

    private static void freeTable(Runnable refreshTableView) {
        String tidStr = JOptionPane.showInputDialog(mainFrame,
            "Enter Table ID to free:");
        if (tidStr == null) return;

        tidStr = tidStr.trim();

        if (!tidStr.matches("\\d+")) {
            JOptionPane.showMessageDialog(mainFrame, "Table
            ID must be a number!");
            return;
        }
    }

```

```

    }

    try {
        int tid = Integer.parseInt(tidStr);
        if (tid <= 0) {
            JOptionPane.showMessageDialog(mainFrame,
                "Table ID must be a positive number!");
            return;
        }

        if (tid > tables.size()) {
            JOptionPane.showMessageDialog(mainFrame,
                "Invalid Table ID! Maximum table ID is " + tables.size());
            return;
        }

        RestaurantTable table = tables.get(tid - 1);
        if (table.isAvailable) {
            JOptionPane.showMessageDialog(mainFrame,
                "Table " + tid + " is already free!");
            return;
        }

        table.freeTable();

        // Cancel reservations using this table
        ArrayList<Reservation> toRemove = new ArrayList<>();
        for (Reservation res : reservations) {
            for (RestaurantTable t : res.allocatedTables) {
                if (t.table_id == tid) {

```

```

        try {
ReservationDAO.cancelReservation(res.reservationId);
            toRemove.add(res);
        } catch (SQLException e) {
            System.err.println("Error cancelling
reservation: " + e.getMessage());
        }
        break;
    }
}
}
reservations.removeAll(toRemove);

JOptionPane.showMessageDialog(mainFrame, "✓
Table " + tid + " is now free.");
refreshTableView.run();

} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(mainFrame, "Invalid
number format!");
} catch (Exception e) {
    JOptionPane.showMessageDialog(mainFrame, "An
error occurred: " + e.getMessage());
    e.printStackTrace();
}
}

// ===== REPORTS METHOD
=====

private static void viewReports() {

```



```

StringBuilder sb = new StringBuilder();
sb.append("===== RESTAURANT REPORTS
=====\\n\\n");

sb.append("ORDERS:\\n");
sb.append("Total Orders:
").append(Main.orders.size()).append("\\n");
    int pending = 0, confirmed = 0, preparing = 0, ready = 0,
delivered = 0, cancelled = 0;
    for (Order order : Main.orders) {
        switch(order.getStatus()) {
            case PENDING: pending++; break;
            case CONFIRMED: confirmed++; break;
            case PREPARING: preparing++; break;
            case READY: ready++; break;
            case DELIVERED: delivered++; break;
            case CANCELLED: cancelled++; break;
        }
    }
    sb.append(" - Pending: ").append(pending).append("\\n");
    sb.append(" - Confirmed:
").append(confirmed).append("\\n");
    sb.append(" - Preparing:
").append(preparing).append("\\n");
    sb.append(" - Ready: ").append(ready).append("\\n");
    sb.append(" - Delivered:
").append(delivered).append("\\n");
    sb.append(" - Cancelled:
").append(cancelled).append("\\n\\n");

```

```
sb.append("BILLS:\n");
sb.append("Total Bills:
").append(Main.bills.size()).append("\n");
    double totalRevenue = 0;
    for (Bill bill : Main.bills) {
        totalRevenue += bill.getTotal();
    }
sb.append("Total Revenue:
Rs.").append(String.format("%.2f", totalRevenue)).append("\n\n");
```

```
sb.append("TABLES:\n");
int available = 0, occupied = 0;
for (RestaurantTable table : Main.tables) {
    if (table.isAvailable) available++;
    else occupied++;
}
sb.append(" - Available: ").append(available).append("\n");
sb.append(" - Occupied:
").append(occupied).append("\n\n");
```

```
// NEW: Employee statistics
sb.append("EMPLOYEES:\n");
sb.append("Total Employees:
").append(restaurant.getEmployees().size()).append("\n");
int chefs = 0, waiters = 0, managers = 0;
for (Employee emp : restaurant.getEmployees()) {
    switch(emp.getRole()) {
        case CHEF: chefs++; break;
        case WAITER: waiters++; break;
        case MANAGER: managers++; break;
```

```

    }
}
sb.append(" - Chefs: ").append(chefs).append("\n");
sb.append(" - Waiters: ").append(waiters).append("\n");
sb.append(" - Managers:
").append(managers).append("\n");

JTextArea textArea = new JTextArea(sb.toString());
textArea.setFont(new Font("Monospaced", Font.PLAIN,
12));
JScrollPane scrollPane = new JScrollPane(textArea);
scrollPane.setPreferredSize(new Dimension(600, 450));
JOptionPane.showMessageDialog(mainFrame,
scrollPane, "Restaurant Reports",
JOptionPane.INFORMATION_MESSAGE);
}
private static void viewPastEmployees() {
    if (restaurant.getPastEmployees().isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame,
            "No past employees found!",
            "Past Employees",
            JOptionPane.INFORMATION_MESSAGE);
        return;
    }
    // Create custom panel with table
    JPanel panel = new JPanel(new BorderLayout(10,
10));
    panel.setBorder(BorderFactory.createEmptyBorder(10,
10, 10, 10));

```

```

        JLabel header = new JLabel("  Past Employees
Record", SwingConstants.CENTER);
        header.setFont(new Font("Segoe UI", Font.BOLD,
18));
        header.setBorder(BorderFactory.createEmptyBorder(0,
0, 10, 0));
        panel.add(header, BorderLayout.NORTH);
        String[] columns = {"ID", "Name", "Role", "Last Action"};
        DefaultTableModel model = new
DefaultTableModel(columns, 0);
        JTable table = new JTable(model);
        table.setRowHeight(25);
        table.setFont(new Font("SansSerif", Font.PLAIN, 13));
        table.getTableHeader().setFont(new Font("SansSerif",
Font.BOLD, 14));
        table.getTableHeader().setBackground(new Color(149,
165, 166));
        table.getTableHeader().setForeground(Color.WHITE);
        for (Employee emp : restaurant.getPastEmployees()) {
            model.addRow(new Object[]{
                emp.getId(),
                emp.getName(),
                emp.getRole(),
                emp.getAction()
            });
        }
        JScrollPane scrollPane = new JScrollPane(table);
        scrollPane.setPreferredSize(new Dimension(600,
300));
        panel.add(scrollPane, BorderLayout.CENTER);

```

```

        JLabel footer = new JLabel("Total Past Employees: " +
restaurant.getPastEmployees().size(),
SwingConstants.CENTER);
        footer.setFont(new Font("SansSerif", Font.ITALIC,
12));
        footer.setBorder(BorderFactory.createEmptyBorder(10,
0, 0, 0));
        panel.add(footer, BorderLayout.SOUTH);
        JOptionPane.showMessageDialog(mainFrame, panel,
"Past Employees", JOptionPane.PLAIN_MESSAGE);
    }
    // ===== VIEW DAILY REVENUE
METHOD =====
    private static void viewDailyRevenue() {
        // Create custom styled panel
        JPanel panel = new JPanel(new BorderLayout(15,
15));
        panel.setBorder(BorderFactory.createEmptyBorder(20,
20, 20, 20));
        panel.setBackground(new Color(236, 240, 241));
        // Header
        JLabel header = new JLabel("💰 Daily Revenue
Report", SwingConstants.CENTER);
        header.setFont(new Font("Segoe UI", Font.BOLD,
24));
        header.setForeground(new Color(44, 62, 80));
        header.setBorder(BorderFactory.createEmptyBorder(0,
0, 15, 0));
        panel.add(header, BorderLayout.NORTH);
        // Main content area

```

```

        JPanel contentPanel = new JPanel(new GridLayout(0,
1, 10, 10));
        contentPanel.setBackground(Color.WHITE);

contentPanel.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(new Color(189, 195,
199), 2),
        BorderFactory.createEmptyBorder(20, 20, 20, 20)
));
// Calculate revenue statistics
double totalRevenue = 0;
int totalOrders = Main.orders.size();
int completedOrders = 0;
int pendingOrders = 0;
Map<String, Double> categoryRevenue = new
HashMap<>();
    for (Bill bill : Main.bills) {
        totalRevenue += bill.getTotal();
        Order order = bill.getOrder();
        if (order.getStatus() == OrderStatus.DELIVERED) {
            completedOrders++;
        }
// Calculate category-wise revenue
        for (OrderItem item : order.getItems()) {
            String itemName = item.getItemName();
            categoryRevenue.put(itemName,
                categoryRevenue.getOrDefault(itemName, 0.0) +
item.getSubtotal());
        }
    }

```

```

        for (Order order : Main.orders) {
            if (order.getStatus() != OrderStatus.DELIVERED &&
order.getStatus() != OrderStatus.CANCELLED) {
                pendingOrders++;
            }
        }
        // Add statistics labels
        addRevenueLabel(contentPanel, " Total Revenue:",
"Rs. " + String.format("%.2f", totalRevenue), new Color(39, 174,
96));
        addRevenueLabel(contentPanel, " Total Orders:",
String.valueOf(totalOrders), new Color(52, 152, 219));
        addRevenueLabel(contentPanel, " Completed
Orders:", String.valueOf(completedOrders), new Color(46, 204,
113));
        addRevenueLabel(contentPanel, " Pending
Orders:", String.valueOf(pendingOrders), new Color(241, 196,
15));
        addRevenueLabel(contentPanel, " Average Order
Value:",
        completedOrders > 0 ? "Rs. " + String.format("%.2f",
totalRevenue / completedOrders) : "Rs. 0.00",
        new Color(155, 89, 182));
        panel.add(contentPanel, BorderLayout.CENTER);
        // Top selling items section
        if (!categoryRevenue.isEmpty()) {
            JPanel topItemsPanel = new JPanel(new
BorderLayout(5, 5));
            topItemsPanel.setBackground(new Color(236, 240,
241));

```

```

topItemsPanel.setBorder(BorderFactory.createEmptyBorder(10,
0, 0, 0));
    JLabel topItemsHeader = new JLabel("🏆 Top Selling
Items:", SwingConstants.LEFT);
    topItemsHeader.setFont(new Font("Segoe UI",
Font.BOLD, 14));
    topItemsPanel.add(topItemsHeader,
BorderLayout.NORTH);
    StringBuilder topItems = new StringBuilder();
    categoryRevenue.entrySet().stream()
        .sorted((e1, e2) -> Double.compare(e2.getValue(),
e1.getValue()))
        .limit(5)
        .forEach(entry -> {
            topItems.append(String.format(" • %s: Rs. %.2f\n",
entry.getKey(), entry.getValue()));
        });
    JTextArea itemsArea = new
JTextArea(topItems.toString());
    itemsArea.setFont(new Font("Monospaced",
Font.PLAIN, 12));
    itemsArea.setEditable(false);
    itemsArea.setBackground(Color.WHITE);

itemsArea.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(new Color(189, 195,
199), 1),
    BorderFactory.createEmptyBorder(10, 10, 10, 10)
));

```



```

        toplItemsPanel.add(itemsArea,
BorderLayout.CENTER);
        panel.add(toplItemsPanel, BorderLayout.SOUTH);
    }
    JOptionPane.showMessageDialog(mainFrame, panel,
"Daily Revenue Report", JOptionPane.PLAIN_MESSAGE);
}

private static void addRevenueLabel(JPanel panel,
String label, String value, Color valueColor) {
    JPanel rowPanel = new JPanel(new BorderLayout(10,
0));

    rowPanel.setBackground(Color.WHITE);

    rowPanel.setBorder(BorderFactory.createEmptyBorder(5, 0, 5,
0));

    JLabel labelComponent = new JLabel(label);
    labelComponent.setFont(new Font("SansSerif",
Font.PLAIN, 14));
    labelComponent.setForeground(new Color(52, 73,
94));

    JLabel valueComponent = new JLabel(value);
    valueComponent.setFont(new Font("SansSerif",
Font.BOLD, 16));
    valueComponent.setForeground(valueColor);
    rowPanel.add(labelComponent, BorderLayout.WEST);
    rowPanel.add(valueComponent, BorderLayout.EAST);
    panel.add(rowPanel);
}

```

```

// ===== CHEF ACTION METHODS
=====

```

```

private static void viewAllOrdersChef() {
    if (Main.orders.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame, "No
orders available!\nTotal orders in system: " + Main.orders.size(),
"Chef Dashboard", JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    StringBuilder sb = new StringBuilder();
    sb.append("===== ALL ORDERS
=====\\n");
    sb.append("Total Orders:
").append(Main.orders.size()).append("\\n\\n");
    for (Order order : Main.orders) {
        sb.append("Order #").append(order.getId())
        .append(" | Table: ").append(order.getTableNo())
        .append(" | Status: ").append(order.getStatus())
        .append(" | Items:
").append(order.getItemCount()).append("\\n");
    }
    JTextArea textArea = new JTextArea(sb.toString());
    textArea.setFont(new Font("Monospaced", Font.PLAIN,
12));
    JScrollPane scrollPane = new JScrollPane(textArea);
    scrollPane.setPreferredSize(new Dimension(500, 300));
    JOptionPane.showMessageDialog(mainFrame,
scrollPane, "All Orders",
JOptionPane.INFORMATION_MESSAGE);
}

private static void updateOrderStatusChef() {

```

```

        if (Main.orders.isEmpty()) {
            JOptionPane.showMessageDialog(mainFrame, "No
orders available!", "Chef Dashboard",
JOptionPane.INFORMATION_MESSAGE);
            return;
        }

        // Show current orders
        StringBuilder orderList = new StringBuilder();
        for (Order order : Main.orders) {
            orderList.append("Order #").append(order.getId())
                .append(" | Table ").append(order.getTableNo())
                .append(" | Status:
").append(order.getStatus()).append("\n");
        }

        String orderIdStr =
JOptionPane.showInputDialog(mainFrame,
        "Current Orders:\n" + orderList + "\nEnter Order ID to
update:");
        if (orderIdStr == null) return;

        try {
            int orderId = Integer.parseInt(orderIdStr);
            Order order = null;
            for (Order o : Main.orders) {
                if (o.getId() == orderId) {
                    order = o;
                    break;
                }
            }

```

```

    }

    if (order == null) {
        JOptionPane.showMessageDialog(mainFrame,
"Order not found!");
        return;
    }

    // Get valid next statuses for chef
    ArrayList<OrderStatus> validStatuses = new
ArrayList<>();
    OrderStatus currentStatus = order.getStatus();

    if (currentStatus == OrderStatus.PENDING) {
        validStatuses.add(OrderStatus.CONFIRMED);
        validStatuses.add(OrderStatus.CANCELLED);
    } else if (currentStatus == OrderStatus.CONFIRMED) {
        validStatuses.add(OrderStatus.PREPARING);
        validStatuses.add(OrderStatus.CANCELLED);
    } else if (currentStatus == OrderStatus.PREPARING) {
        validStatuses.add(OrderStatus.READY);
        validStatuses.add(OrderStatus.CANCELLED);
    } else {
        JOptionPane.showMessageDialog(mainFrame,
"Order is in " + currentStatus + " status. Cannot update from Chef
dashboard.");
        return;
    }

    if (validStatuses.isEmpty()) {

```

```
        JOptionPane.showMessageDialog(mainFrame, "No  
valid status transitions available!");  
        return;  
    }
```

```
        OrderStatus[] statusArray = validStatuses.toArray(new  
OrderStatus[0]);
```

```
        OrderStatus newStatus = (OrderStatus)  
JOptionPane.showInputDialog(  
        mainFrame,  
        "Current Status: " + currentStatus + "\nSelect New  
Status:",  
        "Change Status",  
        JOptionPane.QUESTION_MESSAGE,  
        null,  
        statusArray,  
        statusArray[0]  
    );
```

```
        if (newStatus != null) {  
            boolean changed = order.changeStatus(newStatus);  
            if (changed) {  
                JOptionPane.showMessageDialog(mainFrame,  
                "Status updated to: " + newStatus);  
            } else {  
                JOptionPane.showMessageDialog(mainFrame,  
                "Invalid status transition!");  
            }  
        }  
    }
```

```

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(mainFrame, "Invalid
Order ID!");
    }
}

private static void viewPendingOrdersChef() {
    ArrayList<Order> pendingOrders = new ArrayList<>();
    for (Order order : Main.orders) {
        if (order.getStatus() == OrderStatus.PENDING ||
order.getStatus() == OrderStatus.CONFIRMED ||
order.getStatus() == OrderStatus.PREPARING) {
            pendingOrders.add(order);
        }
    }

    if (pendingOrders.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame, "No
pending orders!", "Chef Dashboard",
JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    StringBuilder sb = new StringBuilder();
    sb.append("===== PENDING ORDERS
=====\\n\\n");
    for (Order order : pendingOrders) {
        sb.append("Order #").append(order.getOrderId())
        .append(" | Table: ").append(order.getTableNo())
        .append(" | Status: ").append(order.getStatus())
        .append("\\nItems:\\n");
    }
}

```

```

        for (OrderItem item : order.getItems()) {
            sb.append(" - ").append(item).append("\n");
        }
        sb.append("\n");
    }
    JTextArea textArea = new JTextArea(sb.toString());
    textArea.setFont(new Font("Monospaced", Font.PLAIN,
12));
    JScrollPane scrollPane = new JScrollPane(textArea);
    scrollPane.setPreferredSize(new Dimension(500, 400));
    JOptionPane.showMessageDialog(mainFrame,
scrollPane, "Pending Orders",
JOptionPane.INFORMATION_MESSAGE);
}
// ===== WAITER ACTION METHODS
=====

private static void viewReadyOrdersWaiter() {
    ArrayList<Order> readyOrders = new ArrayList<>();
    for (Order order : Main.orders) {
        if (order.getStatus() == OrderStatus.READY) {
            readyOrders.add(order);
        }
    }

    if (readyOrders.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame, "No
ready orders!", "Waiter Dashboard",
JOptionPane.INFORMATION_MESSAGE);
        return;
    }

```

```

StringBuilder sb = new StringBuilder();
sb.append("===== READY ORDERS
=====\\n\\n");
for (Order order : readyOrders) {
    sb.append("Order #").append(order.getId())
      .append(" | Table: ").append(order.getTableNo())
      .append(" | Items: ").append(order.getItemCount())
      .append("\\nItems:\\n");
    for (OrderItem item : order.getItems()) {
        sb.append(" - ").append(item).append("\\n");
    }
    sb.append("\\n");
}
JTextArea textArea = new JTextArea(sb.toString());
textArea.setFont(new Font("Monospaced", Font.PLAIN,
12));
JScrollPane scrollPane = new JScrollPane(textArea);
scrollPane.setPreferredSize(new Dimension(500, 400));
JOptionPane.showMessageDialog(mainFrame,
scrollPane, "Ready Orders",
JOptionPane.INFORMATION_MESSAGE);
}

```

```

private static void markAsDeliveredWaiter() {
    ArrayList<Order> readyOrders = new ArrayList<>();
    for (Order order : Main.orders) {
        if (order.getStatus() == OrderStatus.READY) {
            readyOrders.add(order);
        }
    }
}

```



```

    }
    if (readyOrders.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame,
            "No ready orders to deliver!", "Waiter Dashboard",
            JOptionPane.INFORMATION_MESSAGE);
        return;
    }
    StringBuilder orderList = new StringBuilder();
    orderList.append("===== READY ORDERS\n");
    for (Order order : readyOrders) {
        orderList.append("Order\n")
            .append(order.getId())
            .append(" | Table\n")
            .append(order.getTableNo())
            .append(" | Items:\n")
            .append(order.getItemCount()).append("\n");
    }
    String orderIdStr =
        JOptionPane.showInputDialog(mainFrame,
            orderList + "\nEnter Order ID to mark as\n"
            + "delivered:");
    if (orderIdStr == null) return;
    try {
        int orderId = Integer.parseInt(orderIdStr);
        Order order = null;
        for (Order o : Main.orders) {
            if (o.getId() == orderId && o.getStatus()
                == OrderStatus.READY) {
                order = o;
            }
        }
    }

```

```

        break;
    }
}
if (order == null) {
    JOptionPane.showMessageDialog(mainFrame,
"Order not found or not ready!");
    return;
}
boolean changed =
order.changeStatus(OrderStatus.DELIVERED);
if (changed) {
    OrderDAO dao = new OrderDAO();
    dao.updateOrderStatus(order.getDbOrderId(),
order.getStatus());
    JOptionPane.showMessageDialog(mainFrame,
"Order marked as delivered and updated in database!");
} else {
    JOptionPane.showMessageDialog(mainFrame,
"Failed to update status!");
}
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(mainFrame,
"Invalid Order ID!");
}
}

private static void generateBillWaiter() {
    if (Main.orders.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame, "No
orders available!", "Waiter Dashboard",
JOptionPane.INFORMATION_MESSAGE);
    }
}

```

```

        return;
    }

    // Find delivered orders
    ArrayList<Order> deliveredOrders = new ArrayList<>();
    for (Order order : Main.orders) {
        if (order.getStatus() == OrderStatus.DELIVERED) {
            deliveredOrders.add(order);
        }
    }

    if (deliveredOrders.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame, "No
delivered orders available for billing!", "Waiter Dashboard",
JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    StringBuilder orderList = new StringBuilder();
    orderList.append("===== DELIVERED ORDERS
=====\\n");
    for (Order order : deliveredOrders) {
        orderList.append("Order #").append(order.getId())
            .append(" | Table ").append(order.getTableNo())
            .append(" | Total:
Rs.").append(String.format("%.2f",
order.calculateTotal())).append("\\n");
    }

```

```
String orderIdStr =
JOptionPane.showInputDialog(mainFrame,
    orderList + "\nEnter Order ID to generate bill:");
if (orderIdStr == null) return;

try {
    int orderId = Integer.parseInt(orderIdStr);
    Order order = null;
    for (Order o : Main.orders) {
        if (o.getOrderId() == orderId && o.getStatus() ==
OrderStatus.DELIVERED) {
            order = o;
            break;
        }
    }

    if (order == null) {
        JOptionPane.showMessageDialog(mainFrame,
"Order not found or not delivered!");
        return;
    }

    Bill bill = new Bill(order);
    Main.bills.add(bill);

    // Create payment for the bill
    Payment payment = new Payment(bill);
    Main.payments.add(payment);
}
```

```
JOptionPane.showMessageDialog(mainFrame, "Bill  
generated successfully!\nBill ID: " + bill.getBillId() + "\nPayment  
Status: PENDING");
```

```
    } catch (Exception ex) {  
        JOptionPane.showMessageDialog(mainFrame, "Error: "  
+ ex.getMessage());  
    }  
}
```

```
private static void viewBillsWaiter1() {  
    if (Main.bills.isEmpty()) {  
        JOptionPane.showMessageDialog(mainFrame, "No bills  
available!", "Waiter Dashboard",  
JOptionPane.INFORMATION_MESSAGE);  
        return;  
    }  
}
```

```
StringBuilder sb = new StringBuilder();  
sb.append("===== ALL BILLS  
=====\\n\\n");  
for (Bill bill : Main.bills) {  
    // Find payment for this bill  
    Payment payment = null;  
    for (Payment p : Main.payments) {  
        if (p.getBill().getBillId() == bill.getBillId()) {  
            payment = p;  
            break;  
        }  
    }  
}
```

```
String paymentStatus = payment != null ?  
payment.getStatus().toString() : "NO PAYMENT";
```

```
sb.append("Bill #").append(bill.getBillId())  
  .append(" | Order  
#").append(bill.getOrder().getOrderId())  
  .append(" | Table  
").append(bill.getOrder().getTableNo())  
  .append(" | Total: Rs.").append(String.format("%.2f",  
bill.getTotal()))  
  .append(" | Payment:  
").append(paymentStatus).append("\n");  
}
```

```
JTextArea textArea = new JTextArea(sb.toString());  
textArea.setFont(new Font("Monospaced", Font.PLAIN,  
12));  
JScrollPane scrollPane = new JScrollPane(textArea);  
scrollPane.setPreferredSize(new Dimension(700, 300));  
JOptionPane.showMessageDialog(mainFrame,  
scrollPane, "All Bills", JOptionPane.INFORMATION_MESSAGE);  
}
```

```
// NEW: Process Payment method  
private static void processPayment() {  
    if (Main.bills.isEmpty()) {  
        JOptionPane.showMessageDialog(mainFrame, "No bills  
available!", "Payment",  
JOptionPane.INFORMATION_MESSAGE);  
        return;  
    }  
}
```

```

// Find unpaid bills
ArrayList<Bill> unpaidBills = new ArrayList<>();
for (Bill bill : Main.bills) {
    Payment payment = null;
    for (Payment p : Main.payments) {
        if (p.getBill().getBillId() == bill.getBillId()) {
            payment = p;
            break;
        }
    }
    if (payment != null && payment.getStatus() ==
PaymentStatus.UNPAID) {
        unpaidBills.add(bill);
    }
}

if (unpaidBills.isEmpty()) {
    JOptionPane.showMessageDialog(mainFrame, "No
unpaid bills!", "Payment",
JOptionPane.INFORMATION_MESSAGE);
    return;
}

StringBuilder billList = new StringBuilder();
billList.append("===== UNPAID BILLS
=====\\n");
for (Bill bill : unpaidBills) {
    billList.append("Bill #").append(bill.getBillId())

```

```

        .append(" | Order
#").append(bill.getOrder().getOrderId())
        .append(" | Table
").append(bill.getOrder().getTableNo())
        .append(" | Total:
Rs.").append(String.format("%.2f", bill.getTotal()))).append("\n");
    }

```

```

    String billIdStr =
JOptionPane.showInputDialog(mainFrame,
    billList + "\nEnter Bill ID to process payment:");
    if (billIdStr == null) return;

    try {
        int billId = Integer.parseInt(billIdStr);
        Bill bill = null;
        for (Bill b : unpaidBills) {
            if (b.getBillId() == billId) {
                bill = b;
                break;
            }
        }

        if (bill == null) {
            JOptionPane.showMessageDialog(mainFrame, "Bill
not found!");
            return;
        }

        // Find payment for this bill

```



```

Payment payment = null;
for (Payment p : Main.payments) {
    if (p.getBill().getBillId() == bill.getBillId()) {
        payment = p;
        break;
    }
}

if (payment == null) {
    JOptionPane.showMessageDialog(mainFrame,
    "Payment record not found!");
    return;
}

// Select payment method
PaymentMethod[] methods = PaymentMethod.values();
PaymentMethod method = (PaymentMethod)
JOptionPane.showInputDialog(
    mainFrame,
    "Select Payment Method:",
    "Payment Method",
    JOptionPane.QUESTION_MESSAGE,
    null,
    methods,
    methods[0]
);

if (method == null) return;

// Enter amount

```

```

        String amountStr =
JOptionPane.showInputDialog(mainFrame,
        "Bill Total: Rs." + String.format("%.2f", bill.getTotal()) +
        "\nEnter Payment Amount:");
        if (amountStr == null) return;

        double amount = Double.parseDouble(amountStr);

        boolean isPaid = payment.makePayment(method,
amount);

        if (isPaid) {
            JOptionPane.showMessageDialog(mainFrame,
                "✅ Payment Successful!\nBill #" + billId + " is now
PAID");
        } else {
            double remaining = payment.getRemainingBalance();
            JOptionPane.showMessageDialog(mainFrame,
                "Partial payment received.\nRemaining: Rs." +
String.format("%.2f", remaining));
        }

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(mainFrame, "Invalid
number format!");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(mainFrame, "Error: "
+ ex.getMessage());
    }
}
}

```

```

private static void generateBillWaiter1() {
    if (Main.orders.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame, "No
orders available!", "Waiter Dashboard",
JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    // Find delivered orders
    ArrayList<Order> deliveredOrders = new ArrayList<>();
    for (Order order : Main.orders) {
        if (order.getStatus() == OrderStatus.DELIVERED) {
            deliveredOrders.add(order);
        }
    }

    if (deliveredOrders.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame, "No
delivered orders available for billing!", "Waiter Dashboard",
JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    StringBuilder orderList = new StringBuilder();
    orderList.append("===== DELIVERED ORDERS
=====\\n");
    for (Order order : deliveredOrders) {
        orderList.append("Order #").append(order.getId())
            .append(" | Table ").append(order.getTableNo())

```

```
        .append(" | Total:  
Rs.").append(String.format("%.2f",  
order.calculateTotal())).append("\n");  
    }
```

```
    String orderIdStr =  
JOptionPane.showInputDialog(mainFrame,  
    orderList + "\nEnter Order ID to generate bill:");  
    if (orderIdStr == null) return;  
  
    try {  
        int orderId = Integer.parseInt(orderIdStr);  
        Order order = null;  
        for (Order o : Main.orders) {  
            if (o.getOrderId() == orderId && o.getStatus() ==  
OrderStatus.DELIVERED) {  
                order = o;  
                break;  
            }  
        }  
    }  
  
    if (order == null) {  
        JOptionPane.showMessageDialog(mainFrame,  
"Order not found or not delivered!");  
        return;  
    }
```

```
    Bill bill = new Bill(order);  
    Main.bills.add(bill);
```

```

        // Create payment for the bill
        Payment payment = new Payment(bill);
        Main.payments.add(payment);

        JOptionPane.showMessageDialog(mainFrame, "Bill
generated successfully!\nBill ID: " + bill.getBillId() + "\nPayment
Status: PENDING");

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(mainFrame, "Error: "
+ ex.getMessage());
    }
}

private static void viewBillsWaiter() {
    if (Main.bills.isEmpty()) {
        JOptionPane.showMessageDialog(mainFrame, "No bills
available!", "Waiter Dashboard",
JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    StringBuilder sb = new StringBuilder();
    sb.append("===== ALL BILLS
=====\\n\\n");
    for (Bill bill : Main.bills) {
        // Find payment for this bill
        Payment payment = null;
        for (Payment p : Main.payments) {
            if (p.getBill().getBillId() == bill.getBillId()) {
                payment = p;
            }
        }
    }
}

```

```

        break;
    }
}

```

```

String paymentStatus = payment != null ?
payment.getStatus().toString() : "NO PAYMENT";

```

```

sb.append("Bill #").append(bill.getBillId())
.append(" | Order
#").append(bill.getOrder().getOrderId())
.append(" | Table
").append(bill.getOrder().getTableNo())
.append(" | Total: Rs.").append(String.format("%.2f",
bill.getTotal()))
.append(" | Payment:
").append(paymentStatus).append("\n");
}

```

```

JTextArea textArea = new JTextArea(sb.toString());
textArea.setFont(new Font("Monospaced", Font.PLAIN,
12));
JScrollPane scrollPane = new JScrollPane(textArea);
scrollPane.setPreferredSize(new Dimension(700, 300));
JOptionPane.showMessageDialog(mainFrame,
scrollPane, "All Bills", JOptionPane.INFORMATION_MESSAGE);
}

```

```

// ===== CHEF PANEL
=====

```

```

private static JPanel createChefPanel() {
    JPanel panel = new JPanel();
}

```

```
panel.setBackground(new Color(240, 248, 255));  
panel.setLayout(new BorderLayout());
```

```
JLabel label = new JLabel("👨‍🍳 Chef Dashboard",  
SwingConstants.CENTER);  
label.setFont(new Font("Serif", Font.BOLD, 30));  
panel.add(label, BorderLayout.NORTH);
```

```
JPanel buttonPanel = new JPanel();  
buttonPanel.setBackground(new Color(240, 248, 255));  
buttonPanel.setLayout(new GridLayout(2, 2, 15, 15));
```

```
buttonPanel.setBorder(BorderFactory.createEmptyBorder(50, 50,  
50, 50));
```

```
JButton viewOrdersBtn = new JButton("View Orders");  
JButton updateStatusBtn = new JButton("Update Order  
Status");  
JButton viewPendingBtn = new JButton("View Pending  
Orders");  
JButton backBtn = new JButton("Back");
```

```
buttonPanel.add(viewOrdersBtn);  
buttonPanel.add(updateStatusBtn);  
buttonPanel.add(viewPendingBtn);  
buttonPanel.add(backBtn);
```

```
viewOrdersBtn.addActionListener(e ->  
viewAllOrdersChef());
```

```
        updateStatusBtn.addActionListener(e ->
updateOrderStatusChef());
        viewPendingBtn.addActionListener(e ->
viewPendingOrdersChef());
```

```
        backBtn.addActionListener(e ->
cardLayout.show(mainPanel, "ROLE_SELECTION"));
```

```
        panel.add(buttonPanel, BorderLayout.CENTER);
        return panel;
    }
    // ===== WAITER PANEL
    =====
```

```
    private static JPanel createWaiterPanel() {
        JPanel panel = new JPanel();
        panel.setBackground(new Color(255, 250, 240));
        panel.setLayout(new BorderLayout());

        JLabel label = new JLabel("👤 Waiter Dashboard",
SwingConstants.CENTER);
        label.setFont(new Font("Serif", Font.BOLD, 30));
        panel.add(label, BorderLayout.NORTH);
```

```
        JPanel buttonPanel = new JPanel();
        buttonPanel.setBackground(new Color(255, 250, 240));
        buttonPanel.setLayout(new GridLayout(3, 3, 15, 15)); //
Changed to 3x3 grid
```

```
        buttonPanel.setBorder(BorderFactory.createEmptyBorder(50, 50,
50, 50));
```



```

        JButton createOrderBtn = new JButton("Create Order");
        JButton viewReadyBtn = new JButton("View Ready
Orders");
        JButton markDeliveredBtn = new JButton("Mark as
Delivered");
        JButton generateBillBtn = new JButton("Generate Bill");
        JButton viewBillsBtn = new JButton("View Bills");
        JButton processPaymentBtn = new JButton("Process
Payment"); // NEW
        JButton backBtn = new JButton("Back");

        buttonPanel.add(createOrderBtn);
        buttonPanel.add(viewReadyBtn);
        buttonPanel.add(markDeliveredBtn);
        buttonPanel.add(generateBillBtn);
        buttonPanel.add(viewBillsBtn);
        buttonPanel.add(processPaymentBtn); // NEW
        buttonPanel.add(backBtn);
        buttonPanel.add(new JLabel()); // Empty space
        buttonPanel.add(new JLabel()); // Empty space

        createOrderBtn.addActionListener(e ->
cardLayout.show(mainPanel, "ORDER_MANAGEMENT"));
        viewReadyBtn.addActionListener(e ->
viewReadyOrdersWaiter());
        markDeliveredBtn.addActionListener(e ->
markAsDeliveredWaiter());
        generateBillBtn.addActionListener(e ->
generateBillWaiter());

```

```
viewBillsBtn.addActionListener(e -> viewBillsWaiter1());
processPaymentBtn.addActionListener(e ->
processPayment()); // NEW
```

```
backBtn.addActionListener(e ->
cardLayout.show(mainPanel, "ROLE_SELECTION"));
```

```
panel.add(buttonPanel, BorderLayout.CENTER);
return panel;
}
// ===== ORDER MANAGEMENT
PANEL =====
private static JPanel createOrderManagementPanel() {
    JPanel panel = new JPanel(new BorderLayout(10,
10));
```

```
panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10,
10));
```

```
JLabel titleLabel = new JLabel("Order Management",
JLabel.CENTER);
titleLabel.setFont(new Font("Arial", Font.BOLD, 24));
panel.add(titleLabel, BorderLayout.NORTH);
```

```
// Menu display
String[] columns = {"ID", "Name", "Category",
"total_amount"};
DefaultTableModel menuModel = new
DefaultTableModel(columns, 0);
```

```

JTable menuTable = new JTable(menuModel);

menuTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

Runnable refreshMenuTable = () -> {
    menuModel.setRowCount(0);
    try (Connection con =
DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC",
        "root", "Angela@3root");
        PreparedStatement ps =
con.prepareStatement("SELECT * FROM menu_items");
        ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            int id = rs.getInt("item_id");
            String name = rs.getString("name");
            String category = rs.getString("category");
            double totalAmount =
rs.getDouble("total_amount");
            boolean available =
rs.getBoolean("is_available");
            // Add only available items
            if (available) {
                menuModel.addRow(new Object[]{
                    id, name, category, String.format("₹
%.2f", totalAmount)
                });
            }
        }
    }
}

```

```

    }
}
System.out.println("✅ Menu items loaded
successfully into Order Management panel.");
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(panel,
        "Error loading menu items: " +
ex.getMessage(),
        "Database Error",
JOptionPane.ERROR_MESSAGE);
}
};
// Automatically load items when the panel opens
SwingUtilities.invokeLater(refreshMenuTable);

JScrollPane menuScrollPane = new
JScrollPane(menuTable);

menuScrollPane.setBorder(BorderFactory.createTitledBorder("Av
ailable Menu Items"));
panel.add(menuScrollPane, BorderLayout.CENTER);

// Buttons panel
JPanel buttonPanel = new JPanel(new GridLayout(2,
3, 10, 10));

JButton addToOrderBtn = new JButton("Add to
Order");
JButton createOrderBtn = new JButton("Create New
Order");

```

```

        JButton finishOrderBtn = new JButton("Finish
Order");

        JButton viewCurrentBtn = new JButton("View Current
Items");

        JButton refreshBtn = new JButton("Refresh Menu");
        JButton backBtn = new JButton("Back to Waiter");


        buttonPanel.add(addToOrderBtn);
        buttonPanel.add(createOrderBtn);
        buttonPanel.add(finishOrderBtn);
        buttonPanel.add(viewCurrentBtn);
        buttonPanel.add(refreshBtn);
        buttonPanel.add(backBtn);


        panel.add(buttonPanel, BorderLayout.SOUTH);


        // Store current order being created
        final Order[] currentOrder = {null};


        createOrderBtn.addActionListener(e -> {
            String tableStr =
JOptionPane.showInputDialog(mainFrame, "Enter Table
Number:");

            if (tableStr != null && !tableStr.trim().isEmpty()) {
                try {
                    int tableNo = Integer.parseInt(tableStr);
                    currentOrder[0] = new Order(tableNo);

JOptionPane.showMessageDialog(mainFrame, "New order

```

```

        created for Table " + tableNo + "\nOrder ID: " +
        currentOrder[0].getOrderId());
    } catch (NumberFormatException ex) {

JOptionPane.showMessageDialog(mainFrame, "Invalid table
number!");
    }
    });

    addToOrderBtn.addActionListener(e -> {
        if (currentOrder[0] == null) {
            JOptionPane.showMessageDialog(mainFrame,
"Please create an order first!");
            return;
        }

        int selectedRow = menuTable.getSelectedRow();
        if (selectedRow < 0) {
            JOptionPane.showMessageDialog(mainFrame,
"Please select an item from the menu!");
            return;
        }

        int itemId = (Integer)
menuModel.getValueAt(selectedRow, 0);
        String itemName = (String)
menuModel.getValueAt(selectedRow, 1);

```

```

        String amountStr =
menuModel.getValueAt(selectedRow, 3).toString().replace("₹",
"").trim();

        double total_amount =
Double.parseDouble(amountStr);

        String qtyStr =
JOptionPane.showInputDialog(mainFrame, "Enter Quantity for " +
itemName + ":");
        if (qtyStr == null) return;

        try {
            int qty = Integer.parseInt(qtyStr);
            if (qty <= 0) {

JOptionPane.showMessageDialog(mainFrame, "Quantity must be
positive!");

                return;
            }

            OrderItem item = new OrderItem(itemName,
total_amount, qty);
            currentOrder[0].addItem(item);
            JOptionPane.showMessageDialog(mainFrame,
"Item added to order!");
            JOptionPane.showMessageDialog(mainFrame,
"Item added to order!");
            //  Immediately update total in database
            try (Connection con =
DriverManager.getConnection(

```

```

"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC",
    "root", "Angela@3root")) {
    String updateSQL = "UPDATE orders SET
total_amount = ? WHERE order_id = ?";
    PreparedStatement ps =
con.prepareStatement(updateSQL);
    ps.setDouble(1,
currentOrder[0].calculateTotal());
    ps.setInt(2, currentOrder[0].getDbOrderId()); //
 Use real DB order ID
    ps.executeUpdate();
    System.out.println(" Updated total_amount in
DB for order_id: " + currentOrder[0].getDbOrderId());
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(mainFrame,
"⚠ Error updating order total: " + ex.getMessage());
    }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(mainFrame,
"Invalid quantity!");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(mainFrame,
"Error: " + ex.getMessage());
    }
});

viewCurrentBtn.addActionListener(e -> {

```



```

        if (currentOrder[0] == null ||
currentOrder[0].getItems().isEmpty()) {
            JOptionPane.showMessageDialog(mainFrame,
"No items in current order!");
            return;
        }

        StringBuilder sb = new StringBuilder();
        sb.append("Order
#").append(currentOrder[0].getOrderId())
        .append(" | Table:
").append(currentOrder[0].getTableNo())
        .append(" | Status:
").append(currentOrder[0].getStatus())
        .append("\n\nItems:\n");
        for (OrderItem item : currentOrder[0].getItems()) {
            sb.append(item).append("\n");
        }
        sb.append("\nSubtotal:
Rs.").append(String.format("%.2f",
currentOrder[0].calculateSubtotal()));
        sb.append("\nTotal:
Rs.").append(String.format("%.2f",
currentOrder[0].calculateTotal()));

        JTextArea textArea = new JTextArea(sb.toString());
        textArea.setFont(new Font("Monospaced",
Font.PLAIN, 12));
        JScrollPane scrollPane = new
JScrollPane(textArea);

```

```
scrollPane.setPreferredSize(new Dimension(500,
300));

JOptionPane.showMessageDialog(mainFrame,
scrollPane, "Current Order",
JOptionPane.INFORMATION_MESSAGE);
});
```

```
finishOrderBtn.addActionListener(e -> {
    if (currentOrder[0] == null ||
currentOrder[0].getItems().isEmpty()) {

JOptionPane.showMessageDialog(mainFrame, "Cannot submit
empty order!");
```

```
        return;
    }
    Main.orders.add(currentOrder[0]);
    try {
        OrderDAO dao = new OrderDAO();
        int dbId = dao.saveOrder(currentOrder[0]);
        currentOrder[0].setDbOrderId(dbId); // ✓
```

link Java object to DB row

```
JOptionPane.showMessageDialog(mainFrame, "Order submitted
and saved! (DB ID: " + dbId + ")");
    } catch (Exception ex) {
```

```
JOptionPane.showMessageDialog(mainFrame, "Error saving
order: " + ex.getMessage());
    }
});
```

```

        refreshBtn.addActionListener(e ->
refreshMenuTable.run());
        backBtn.addActionListener(e ->
cardLayout.show(mainPanel, "WAITER"));

```

```

        return panel;
    }
}

```

// ===== EMPLOYEE CLASSES (NEW)

=====

```

abstract class Employee {
    protected int id;
    protected String name;
    protected EmployeeRole role;
    private static int counter = 1;
    public Employee(String name, EmployeeRole role) {
        this.id = counter++;
        this.name = name;
        this.role = role;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public EmployeeRole getRole() { return role; }
    public abstract String getAction();
}

class Chef extends Employee {
    public Chef(String name) { super(name,
EmployeeRole.CHEF); }
    public String getAction() { return "👨🍳 Preparing orders"; }
}

```

```

    }
    class Waiter extends Employee {
        public Waiter(String name) { super(name,
EmployeeRole.WAITER); }
        public String getAction() { return "👤 Serving customers"; }
    }
    class Manager extends Employee {
        public Manager(String name) { super(name,
EmployeeRole.MANAGER); }
        public String getAction() { return "👔 Overseeing
restaurant operations"; }
    }
    // ===== EMPLOYEE ROLE ENUM
(NEW) =====
    enum EmployeeRole { CHEF, WAITER, MANAGER }
    // ===== RESTAURANT CLASS (NEW)
=====

    class Restaurant {
        String name;
        ArrayList<Employee> employees = new ArrayList<>();
        ArrayList<Employee> pastEmployees = new
ArrayList<>();
        public Restaurant(String name) { this.name = name; }
        public void addEmployee(Employee e) {
employees.add(e); }
        public void addPastEmployee(Employee e) {
pastEmployees.add(e); }
        public ArrayList<Employee> getEmployees() { return
employees; }

```

```

        public ArrayList<Employee> getPastEmployees() {
return pastEmployees; }
        public Employee findEmployee(int id) {
        for (Employee e : employees)
        if (e.getId() == id) return e;
        return null;
        }
    }
}

```

```

// ===== EXISTING CLASSES
=====

```

```

class Menu {

    private List<MenuItem> items = new ArrayList<>();

    public void addItem(MenuItem item) {
        String sql = "INSERT INTO menu_items (item_id,
name, category, total_amount, is_available) VALUES (?, ?, ?, ?,
?)";

        try (Connection conn =
DatabaseIntegration.getConnection();
            PreparedStatement ps =
conn.prepareStatement(sql)) {
            // ✓ Important: make sure the DB writes data
instantly

            conn.setAutoCommit(true);
            ps.setInt(1, item.getId());
            ps.setString(2, item.getName());
            ps.setString(3, item.getCategory());
            ps.setDouble(4, item.getTotal_amount());
            ps.setBoolean(5, item.isAvailable());

```

```

        ps.executeUpdate();
        System.out.println("✅ Menu item saved
successfully!");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

    public ArrayList<MenuItem> getAllItems() {
        ArrayList<MenuItem> items = new ArrayList<>();
        String sql = "SELECT item_id, name, category,
total_amount, is_available FROM menu_items";
        try (Connection conn =
DatabaseIntegration.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                int id = rs.getInt("item_id");
                String name = rs.getString("name");
                String category = rs.getString("category");
                double total_amount =
rs.getDouble("total_amount");
                boolean available =
rs.getBoolean("is_available");
                items.add(new MenuItem(id, name, category,
total_amount, available));
            }
            System.out.println("✅ Loaded " + items.size() + "
menu items from database.");
        }
    }
}

```

```
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return items;  
}
```

```
public List<MenuItem> getItems(){  
    return items;  
}  
public List<MenuItem> searchByName(String name) {  
    return items.stream()  
        .filter(i ->  
i.getName().toLowerCase().contains(name.toLowerCase()))  
        .toList();  
}  
public List<MenuItem> filterByCategory(String category) {  
    return items.stream()  
        .filter(i ->  
i.getCategory().equalsIgnoreCase(category))  
        .toList();  
}  
public List<MenuItem> filterByAvailability(boolean  
available) {  
    return items.stream()  
        .filter(i -> i.isAvailable() == available)  
        .toList();  
}  
}
```

```

class MenuItem {
    int id;
    String name;
    String category;
    double total_amount;
    boolean available;
    List<String> ingredients;
    boolean isAvailable;
    MenuItem(int id, String name, String category, double
total_amount, boolean isAvailable) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.total_amount = total_amount;
        this.isAvailable = isAvailable;
    }
    public MenuItem(int id2, String name2, String category2,
double total_amount2, List<String> ingredients2) {
        // TODO Auto-generated constructor stub
    }
    public void toggleAvailability() {
        available = !available;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public String getCategory() { return category; }
    public boolean isAvailable() { return available; }
    public double getTotal_amount() { return total_amount; }
}
class InventoryUI {

```



```

private Inventory inventory;
private JTable ingredientTable;
private DefaultTableModel tableModel;
private JPanel contentPanel;

private void restockIngredient() {
    List<Ingredient> allIngredients =
inventory.getAllIngredients();
    if (allIngredients.isEmpty()) {

        JOptionPane.showMessageDialog(Main.mainFrame, "No
ingredients available to restock.");
        return;
    }
    String[] names =
allIngredients.stream().map(Ingredient::getName).toArray(String[]:
new);

    JComboBox<String> ingredientDropdown = new
JComboBox<>(names);
    JTextField quantityField = new JTextField();
    Object[] fields = {
        "Select Ingredient:", ingredientDropdown,
        "Quantity to add:", quantityField
    };
    int option =
JOptionPane.showConfirmDialog(Main.mainFrame, fields,
"Restock Ingredient", JOptionPane.OK_CANCEL_OPTION);
    if (option == JOptionPane.OK_OPTION) {
        try {

```

```

        String selectedName = (String)
ingredientDropDown.getSelectedItemAt();
        double addQuantity =
Double.parseDouble(quantityField.getText().trim());
        if (addQuantity <= 0) throw new
Exception("Quantity must be positive.");
        Ingredient ing =
inventory.getIngredient(selectedName);
        double newQuantity = ing.getQuantity() +
addQuantity;
        if (newQuantity > ing.getThreshold()) {
            throw new Exception("Cannot restock.
Quantity exceeds threshold of " + ing.getThreshold());
        }
        // ✓ Update in-memory object
        ing.setQuantity(newQuantity);
        // ✓ Update database
        try (Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/spice_
garden?useSSL=false&allowPublicKeyRetrieval=true&serverTime
zone=UTC", "root", "Angela@3root");
            PreparedStatement ps =
con.prepareStatement("UPDATE ingredients SET quantity = ?
WHERE name = ?")) {
            ps.setDouble(1, newQuantity);
            ps.setString(2, selectedName);
            int rows = ps.executeUpdate();
            if (rows > 0) {

```

```
JOptionPane.showMessageDialog(Main.mainFrame, "Ingredient  
restocked successfully!");
```

```
    } else {
```

```
JOptionPane.showMessageDialog(Main.mainFrame, "Error:  
Ingredient not found in database.");
```

```
    }
```

```
    } catch (SQLException e) {
```

```
JOptionPane.showMessageDialog(Main.mainFrame, "Database  
error: " + e.getMessage());
```

```
    }
```

```
    refreshTable();
```

```
    } catch (NumberFormatException ex) {
```

```
JOptionPane.showMessageDialog(Main.mainFrame, "Quantity  
must be numeric.");
```

```
    } catch (Exception ex) {
```

```
JOptionPane.showMessageDialog(Main.mainFrame,  
ex.getMessage());
```

```
    }
```

```
    }
```

```
    }
```

```
    private void deleteIngredient() {
```

```
        int selectedRow =
```

```
ingredientTable.getSelectedRow();
```

```
        if (selectedRow >= 0) {
```

```

        String ingredientName = (String)
tableModel.getValueAt(selectedRow, 0);
        int confirm =
JOptionPane.showConfirmDialog(Main.mainFrame,
        "Are you sure you want to delete " +
ingredientName + "?",
        "Confirm Deletion",
JOptionPane.YES_NO_OPTION);
        if (confirm == JOptionPane.YES_OPTION) {
            //1 Delete from in-memory list
            inventory.ingredients.remove(ingredientName);
            //2 Delete from database

deleteIngredientFromDatabase(ingredientName);
            //3 Refresh UI table
            refreshTable();

JOptionPane.showMessageDialog(Main.mainFrame, "Ingredient
deleted successfully!");
        }
        } else {

JOptionPane.showMessageDialog(Main.mainFrame, "Please
select an ingredient to delete.");
        }
    }

    private void deleteIngredientFromDatabase(String
ingredientName) {

```

```

        String url =
"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC";
        String user = "root";
        String password = "Angela@3root";
        String sql = "DELETE FROM INGREDIENTS
WHERE name = ?";
        try (Connection con =
DriverManager.getConnection(url, user, password);
            PreparedStatement ps =
con.prepareStatement(sql)) {
            ps.setString(1, ingredientName);
            int rowsAffected = ps.executeUpdate();
            if (rowsAffected > 0) {
                System.out.println("✅ Ingredient " +
ingredientName + " deleted from database.");
            } else {
                System.out.println("⚠️ Ingredient not found in
database.");
            }
        } catch (SQLException e) {
            System.err.println("❌ Database deletion error: " +
e.getMessage());

            JOptionPane.showMessageDialog(Main.mainFrame,
                "Error deleting ingredient from database:\n" +
e.getMessage(),
                "Database Error",
                JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

```

    }
    public InventoryUI(Inventory inventory) {
        this.inventory = inventory;
        this.contentPanel = new JPanel(new BorderLayout(10,
10));

        String[] columnNames = {"Name", "Quantity",
"Threshold", "Unit"};
        tableModel = new DefaultTableModel(columnNames, 0);
        ingredientTable = new JTable(tableModel);

ingredientTable.setSelectionMode(ListSelectionModel.SINGLE_S
ELECTION);
        JScrollPane scrollPane = new
JScrollPane(ingredientTable);
        contentPanel.add(scrollPane, BorderLayout.CENTER);

        JPanel buttonPanel = new JPanel();
        JButton addButton = new JButton("Add Ingredient");
        addButton.addActionListener(e ->
inventory.addIngredientGUI(null));

        JButton viewButton = new JButton("View Ingredients");
        viewButton.addActionListener(e -> refreshTable());

        JButton restockButton = new JButton("Restock
Ingredient");
        restockButton.addActionListener(e ->
restockIngredient());

```

```

        JButton deleteButton = new JButton("Delete Ingredient");
        deleteButton.addActionListener(e -> deleteIngredient());

        buttonPanel.add(addButton);
        buttonPanel.add(viewButton);
        buttonPanel.add(restockButton);
        buttonPanel.add(deleteButton);
        contentPanel.add(buttonPanel, BorderLayout.SOUTH);

        refreshTable();

        JButton backButton = new JButton("Back");
        backButton.addActionListener(e ->
Main.cardLayout.show(Main.mainPanel, "MANAGER"));
        buttonPanel.add(backButton);
    }
    public JPanel getContentPane() {
        return contentPanel;
    }
    private void refreshTable() {
        tableModel.setRowCount(0);
        List<Ingredient> ingredients =
inventory.getAllIngredients();
        for (Ingredient ing : ingredients) {
            tableModel.addRow(new Object[]{
                ing.getName(),
                ing.getQuantity(),
                ing.getThreshold(),
                ing.getUnit()
            });
        }
    }

```

```

    }
}
}
class Inventory {
    public Map<String, Ingredient> ingredients = new
HashMap<>();
    public void alertLowStock() {
        for (Ingredient ing : ingredients.values()) {
            if (ing.isLowStock()) {
                System.out.println(" ⚠ Low stock: " +
ing.getName());
            }
        }
    }
    public void addIngredientGUI(JDialog parent) {
        JTextField nameField = new JTextField();
        JTextField quantityField = new JTextField();
        JTextField thresholdField = new JTextField();
        JTextField unitField = new JTextField();
        boolean adding = true;
        while (adding) {
            Object[] fields = {
                "Name:", nameField,
                "Quantity:", quantityField,
                "Threshold:", thresholdField,
                "Unit:", unitField
            };
            int option =
JOptionPane.showConfirmDialog(parent, fields, "Add New
Ingredient", JOptionPane.OK_CANCEL_OPTION);

```



```

        if (option != JOptionPane.OK_OPTION) {
            break;
        }
        try {
            String name = nameField.getText().trim();
            if (name.isEmpty() || name.matches("\\d+")) {
                throw new Exception("Name must be
non-empty and non-numeric.");
            }
            if (ingredients.containsKey(name)) {
                throw new Exception("Ingredient already
exists.");
            }
            double quantity =
Double.parseDouble(quantityField.getText().trim());
            double threshold =
Double.parseDouble(thresholdField.getText().trim());
            String unit = unitField.getText().trim();
            if (unit.isEmpty()) throw new Exception("Unit
cannot be empty.");
            if (quantity >= threshold) {
                throw new Exception("Quantity must be less
than threshold.");
            }
            Ingredient ing = new Ingredient(name, quantity,
unit, threshold);
            ingredients.put(name, ing);
            //  INSERT into database
            insertIngredientToDatabase(ing);

```

```

        int nextOption =
JOptionPane.showOptionDialog(parent,
        "Ingredient added successfully! What
next?",
        "Add Ingredient",
        JOptionPane.YES_NO_OPTION,
        JOptionPane.INFORMATION_MESSAGE,
        null,
        new String[]{"Add Another Ingredient",
"Back"},
        "Add Another Ingredient");
    if (nextOption == JOptionPane.YES_OPTION) {
        nameField.setText("");
        quantityField.setText("");
        thresholdField.setText("");
        unitField.setText("");
    } else {
        adding = false;
    }
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(parent,
"Quantity and Threshold must be numeric.");
} catch (Exception ex) {
    JOptionPane.showMessageDialog(parent,
ex.getMessage());
}
}
}

private void insertIngredientToDatabase(Ingredient ing) {

```

```

        String url =
"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC";
        String user = "root";
        String password = "Angela@3root";
        String sql = "INSERT INTO INGREDIENTS (name,
quantity, unit, threshold) "
            + "VALUES (?, ?, ?, ?)";
        try (Connection con =
DriverManager.getConnection(url, user, password);
            PreparedStatement ps =
con.prepareStatement(sql)) {
            ps.setString(1, ing.getName());
            ps.setDouble(2, ing.getQuantity());
            ps.setString(3, ing.getUnit());
            ps.setDouble(4, ing.getThreshold());
            int rows = ps.executeUpdate();
            if (rows > 0) {
                System.out.println("✅ Ingredient added to
database: " + ing.getName());
            }
        } catch (SQLException e) {
            System.err.println("❌ Error inserting ingredient: "
+ e.getMessage());

            JOptionPane.showMessageDialog(Main.mainFrame,
                "Database error: " + e.getMessage(),
                "Error",
                JOptionPane.ERROR_MESSAGE);
        }
    }

```

```

    }

    public Ingredient getIngredient(String name) {
        return ingredients.get(name);
    }

    public List<Ingredient> getAllIngredients() {
        List<Ingredient> ingredientList = new ArrayList<>();
        String url =
"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC";
        String user = "root";
        String password = "Angela@3root";
        String sql = "SELECT name, quantity, threshold, unit
FROM INGREDIENTS";
        try (Connection con =
DriverManager.getConnection(url, user, password);
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                Ingredient ing = new Ingredient(
                    rs.getString("name"),
                    rs.getDouble("quantity"),
                    rs.getString("unit"),
                    rs.getDouble("threshold")
                );
                ingredientList.add(ing);
            }
        } catch (SQLException e) {
            System.err.println("❌ Error fetching ingredients: "
+ e.getMessage());
        }
    }
}

```

```

        }
        return ingredientList;
    }
}

class Ingredient {
    private String name;
    private double quantity;
    private double threshold;
    private String unit;
    public String getName() { return name; }
    public double getQuantity() { return quantity; }
    public void setQuantity(double quantity) { this.quantity =
quantity; }
    public double getThreshold() { return threshold; }
    public String getUnit() { return unit; }
    public Ingredient() {}
    public Ingredient(String name, double quantity, String unit,
double threshold) {
        this.name = name;
        this.quantity = quantity;
        this.unit = unit;
        this.threshold = threshold;
    }
    public void consume(double amount) {
        if (amount > quantity)
            throw new IllegalArgumentException("Insufficient
stock for " + name);
        quantity -= amount;
    }
    public boolean isLowStock() {

```

```

        return quantity < threshold;
    }
}

class Customer {
    int customerId;
    String name;
    String phone;
    String email;
    Customer(int id, String name, String phone, String email) {
        this.customerId = id;
        this.name = name;
        this.phone = phone;
        this.email = email;
    }
}

class RestaurantTable {
    int table_id;
    int capacity;
    boolean isAvailable;

    RestaurantTable(int id, int cap) {
        this.table_id = id;
        this.capacity = cap;
        this.isAvailable = true;
    }

    void occupyTable() {
        isAvailable = false;
        try {

```

```

        RestaurantTableDAO.updateTableStatus(table_id,
false);
    } catch (SQLException e) {
        System.err.println("Error updating table status: " +
e.getMessage());
    }
}

```

```

void freeTable() {
    isAvailable = true;
    try {
        RestaurantTableDAO.updateTableStatus(table_id,
true);
    } catch (SQLException e) {
        System.err.println("Error updating table status: " +
e.getMessage());
    }
}

```

```

String getStatus() { return isAvailable ? "Available" :
"Occupied"; }
}

```

```

class Reservation {
    int reservationId;
    Customer customer;
    ArrayList<RestaurantTable> allocatedTables;
    String dateTime;
    Reservation(int id, Customer c,
ArrayList<RestaurantTable> tables, String dt) {
        this.reservationId = id;
    }
}

```

```

        this.customer = c;
        this.allocatedTables = tables;
        this.dateTime = dt;
    }
}

// ===== ORDER MANAGEMENT
ENUMS =====

enum OrderStatus {
    PENDING, CONFIRMED, PREPARING, READY,
DELIVERED, CANCELLED
}

enum PaymentStatus {
    UNPAID, PAID, REFUNDED, PARTIALLY_REFUNDED
}

enum PaymentMethod {
    CASH, CARD, UPI, WALLET
}

// ===== ORDER ITEM CLASS
=====

class OrderItem {
    private static int counter = 1;
    private int itemId;
    private String itemName;
    private double total_amount;
    private int quantity;
    public OrderItem(String name, double total_amount, int
qty) {
        if (total_amount <= 0 || qty <= 0) {
            throw new IllegalArgumentException("Invalid
total_amount or quantity");

```



```

    }
    this.itemId = counter++;
    this.itemName = name;
    this.total_amount = total_amount;
    this.quantity = qty;
}
public double getSubtotal() {
    return total_amount* quantity;
}
public void updateQuantity(int newQty) {
    if (newQty <= 0) throw new
IllegalArgumentException("Quantity must be positive");
    this.quantity = newQty;
}
public int getItemId() { return itemId; }
public String getItemName() { return itemName; }
public double getTotal_amount() { return total_amount; }
public int getQuantity() { return quantity; }
@Override
public String toString() {
    return itemName + " x" + quantity + " @ Rs." +
total_amount+ " = Rs." + getSubtotal();
}
}
// ===== ORDER CLASS
=====

class Order {
    private static int counter = 1;
    private int orderId;
    private int tableNo;

```

```
private String customerName;
private LocalDateTime time;
private OrderStatus status;
private ArrayList<OrderItem> items;
private double discount;
private double tax;
private String remarks;
public Order(int table) {
    this.orderId = counter++;
    this.tableNo = table;
    this.customerName = null;
    this.time = LocalDateTime.now();
    this.status = OrderStatus.PENDING;
    this.items = new ArrayList<>();
    this.discount = 0.0;
    this.tax = 5.0;
    this.remarks = null;
}
```

```
public Order(int table, String customerName, String
remarks) {
    this(table);
    this.customerName = customerName;
    this.remarks = remarks;
}
```

```
private int dbOrderId;
public void setDbOrderId(int id) { this.dbOrderId = id; }
public int getDbOrderId() { return dbOrderId; }
public void addItem(OrderItem item) {
```

```

        if (status == OrderStatus.CANCELLED || status ==
OrderStatus.DELIVERED) {
            throw new IllegalStateException("Cannot modify
order");
        }
        items.add(item);
    }
    public boolean removeItem(int itemId) {
        if (status == OrderStatus.CANCELLED || status ==
OrderStatus.DELIVERED) {
            throw new IllegalStateException("Cannot modify
order");
        }
        for (int i = 0; i < items.size(); i++) {
            if (items.get(i).getItemId() == itemId) {
                items.remove(i);
                return true;
            }
        }
        return false;
    }
    public double calculateSubtotal() {
        double total = 0;
        for (OrderItem it : items) total += it.getSubtotal();
        return total;
    }
    public double calculateDiscount() {
        return calculateSubtotal() * (discount / 100.0);
    }
    public double calculateTax() {

```

```

    double subtotal = calculateSubtotal();
    double afterDiscount = subtotal - calculateDiscount();
    return afterDiscount * (tax / 100.0);
}

public double calculateTotal() {
    double subtotal = calculateSubtotal();
    double discountAmt = calculateDiscount();
    double taxAmt = calculateTax();
    return subtotal - discountAmt + taxAmt;
}

public boolean changeStatus(OrderStatus newStatus) {
    boolean validChange = false;
    if (status == OrderStatus.PENDING) {
        if (newStatus == OrderStatus.CONFIRMED ||
newStatus == OrderStatus.CANCELLED) validChange = true;
    } else if (status == OrderStatus.CONFIRMED) {
        if (newStatus == OrderStatus.PREPARING ||
newStatus == OrderStatus.CANCELLED) validChange = true;
    } else if (status == OrderStatus.PREPARING) {
        if (newStatus == OrderStatus.READY || newStatus
== OrderStatus.CANCELLED) validChange = true;
    } else if (status == OrderStatus.READY) {
        if (newStatus == OrderStatus.DELIVERED)
validChange = true;
    }
    if (validChange) {
        this.status = newStatus;
        // ✓ Sync to database immediately
        try {
            OrderDAO dao = new OrderDAO();

```

```

        dao.updateOrderStatus(this.dbOrderId,
this.status);
    } catch (Exception e) {
        System.err.println("✗ Could not update order
status in DB: " + e.getMessage());
    }
    return true;
}
return false;
}

public int getOrderId() { return orderId; }
public int getTableNo() { return tableNo; }
public LocalDateTime getTime() { return time; }
public OrderStatus getStatus() { return status; }
public int getItemCount() { return items.size(); }
public ArrayList<OrderItem> getItems() { return items; }
public void setDiscount(double d) { if (d < 0 || d > 100)
throw new IllegalArgumentException("Invalid discount");
this.discount = d; }
public void setTax(double t) { if (t < 0) throw new
IllegalArgumentException("Invalid tax"); this.tax = t; }
public double getDiscount() { return discount; }
public double getTax() { return tax; }
public String getCustomerName() { return
customerName; }
public void setCustomerName(String name) {
this.customerName = name; }
public String getRemarks() { return remarks; }
public void setRemarks(String r) { this.remarks = r; }
}

```

// ===== BILL CLASS

=====

```
class Bill {
    private static int counter = 1;
    private int billId;
    private Order order;
    private LocalDateTime billTime;
    private double subtotal;
    private double discount;
    private double tax;
    private double total;
    public Bill(Order order) {
        if (order.getItemCount() == 0) {
            throw new IllegalArgumentException("Cannot
generate bill for empty order");
        }
        this.billId = counter++;
        this.order = order;
        this.billTime = LocalDateTime.now();
        this.subtotal = order.calculateSubtotal();
        this.discount = order.calculateDiscount();
        this.tax = order.calculateTax();
        this.total = order.calculateTotal();
    }
    public int getBillId() { return billId; }
    public double getTotal() { return total; }
    public double getSubtotal() { return subtotal; }
    public double getDiscount() { return discount; }
    public double getTax() { return tax; }
    public Order getOrder() { return order; }
```

```

        public LocalDateTime getBillTime() { return billTime; }
    }
    // ===== PAYMENT CLASS
    =====

    class Payment {
        private static int counter = 1;
        private int paymentId;
        private Bill bill;
        private PaymentStatus status;
        private ArrayList<Transaction> transactions;
        private double totalPaid;
        private double totalRefunded;
        class Transaction {
            PaymentMethod method;
            double amount;
            LocalDateTime time;
            String transactionId;
            Transaction(PaymentMethod m, double amt) {
                this.method = m;
                this.amount = amt;
                this.time = LocalDateTime.now();
                this.transactionId = "TXN" +
System.currentTimeMillis();
            }
            public PaymentMethod getMethod() { return method; }
            public double getAmount() { return amount; }
            public String getTransactionId() { return transactionId; }
        }

        public LocalDateTime getTime() { return time; }
    }

```

```
public Payment(Bill bill) {
    this.paymentId = counter++;
    this.bill = bill;
    this.status = PaymentStatus.UNPAID;
    this.transactions = new ArrayList<>();
    this.totalPaid = 0.0;
    this.totalRefunded = 0.0;
}

public boolean makePayment(PaymentMethod method,
double amount) {
    if (status == PaymentStatus.PAID) {
        throw new IllegalStateException("Already paid");
    }
    if (amount <= 0) throw new
IllegalArgumentException("Invalid amount");
    double remaining = bill.getTotal() - totalPaid +
totalRefunded;
    if (amount > remaining + 0.01) {
        throw new IllegalArgumentException("Amount
exceeds bill total");
    }
    Transaction t = new Transaction(method, amount);
    transactions.add(t);
    totalPaid += amount;
    if (totalPaid - totalRefunded >= bill.getTotal() - 0.01) {
        status = PaymentStatus.PAID;
        return true;
    }
    return false;
}
```



```

    public boolean giveRefund(double amount) {
        if (status == PaymentStatus.UNPAID) throw new
IllegalStateException("Nothing to refund");
        if (amount <= 0) throw new
IllegalArgumentException("Invalid refund amount");
        double maxRefund = totalPaid - totalRefunded;
        if (amount > maxRefund + 0.01) throw new
IllegalArgumentException("Refund exceeds paid amount");
        totalRefunded += amount;
        if (totalRefunded >= totalPaid - 0.01) status =
PaymentStatus.REFUNDED;
        else status =
PaymentStatus.PARTIALLY_REFUNDED;
        return true;
    }

    public double getRemainingBalance() {
        double remaining = bill.getTotal() - totalPaid +
totalRefunded;
        return remaining < 0 ? 0 : remaining;
    }

    public int getPaymentId() { return paymentId; }
    public PaymentStatus getStatus() { return status; }
    public double getTotalPaid() { return totalPaid; }
    public double getTotalRefunded() { return totalRefunded; }
}

    public Bill getBill() { return bill; }
    public ArrayList<Transaction> getTransactions() { return
transactions; }
}

```

```

// ===== JDBC / DAO LAYER
=====

class DBConfig {
    // FIXED: Standardized database name
    public static final String DB_URL =
"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&server
Timezone=UTC";
    public static final String DB_USER = "root";
    public static final String DB_PASS = "Angela@3root";
}

class DBConnection {
    public static Connection getConnection() throws
SQLException {
        return
DriverManager.getConnection(DBConfig.DB_URL,
DBConfig.DB_USER, DBConfig.DB_PASS);
    }
}

class DatabaseIntegration {
    private static final String URL =
"jdbc:mysql://localhost:3306/spice_garden?useSSL=false&allowP
ublicKeyRetrieval=true&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD =
"Angela@3root";
    public static Connection getConnection() throws
SQLException {
        Connection conn = DriverManager.getConnection(URL,
USER, PASSWORD);
    }
}

```

```

        conn.setAutoCommit(true); //  every statement saves
immediately
        return conn;
    }
    public static void initializeIngredientsTable() {
        String checkSQL = "SELECT COUNT(*) FROM
ingredients";
        String insertSQL = ""
            INSERT INTO ingredients (name, quantity, unit,
reorder_level, threshold) VALUES
            ('Tomato', 5.0, 'kg', 2.0, 2.0),
            ('Cheese', 1.5, 'kg', 0.5, 1.0),
            ('Flour', 8.0, 'kg', 3.0, 3.0),
            ('Oil', 4.0, 'litre', 2.0, 2.0)
        "";
        try (Connection conn = getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(checkSQL)) {
            if (rs.next() && rs.getInt(1) == 0) {
                stmt.executeUpdate(insertSQL);
                System.out.println(" Sample ingredients
inserted successfully!");
            } else {
                System.out.println(" Ingredients table already
has data. Skipping sample insert.");
            }
        } catch (SQLException e) {
            System.err.println(" Error initializing ingredients: "
+ e.getMessage());
        }
    }

```

```
}  
}
```

```
class OrderDAO {  
    public int saveOrder(Order order) {  
        String sql = "INSERT INTO orders (table_no, status,  
order_time, total_amount) VALUES (?, ?, ?, ?)";  
        try (Connection conn =  
DatabaseIntegration.getConnection();  
            PreparedStatement ps =  
conn.prepareStatement(sql,  
Statement.RETURN_GENERATED_KEYS)) {  
            ps.setInt(1, order.getTableNo());  
            ps.setString(2, order.getStatus().name());  
            ps.setTimestamp(3,  
Timestamp.valueOf(LocalDateTime.now()));  
            ps.setDouble(4, order.calculateTotal());  
            ps.executeUpdate();  
            // ✓ Get the generated order ID and store it  
            ResultSet rs = ps.getGeneratedKeys();  
            if (rs.next()) {  
                int generatedId = rs.getInt(1);  
                order.setDbOrderId(generatedId);  
                System.out.println("ID New Order saved. DB ID  
= " + generatedId);  
            }  
            return order.getDbOrderId();  
        } catch (SQLException e) {  
            System.err.println("✗ Error saving order: " +  
e.getMessage());  
        }  
    }  
}
```

```

        return -1;
    }
}

    public void updateOrderStatus(int orderId,
OrderStatus newStatus) {
    String sql = "UPDATE orders SET status = ? WHERE
order_id = ?";
    try (Connection conn =
DatabaseIntegration.getConnection();
        PreparedStatement ps =
conn.prepareStatement(sql)) {
        ps.setString(1, newStatus.name());
        ps.setInt(2, orderId);
        ps.executeUpdate();
        System.out.println("✅ Order #" + orderId + "
status updated to " + newStatus);
    } catch (SQLException e) {
        System.err.println("❌ Error updating order status:
" + e.getMessage());
    }
}

    class BillDAO {
        public int saveBill(Bill bill, int dbOrderId) throws
SQLException {
            String sql = "INSERT INTO Bills (order_id, bill_time,
subtotal, discount, tax, total) VALUES (?, ?, ?, ?, ?, ?)";
            try (Connection conn = DBConnection.getConnection());

```

```

        PreparedStatement ps =
conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
    ps.setInt(1, dbOrderId);
    ps.setTimestamp(2,
Timestamp.valueOf(bill.getBillTime()));
    ps.setDouble(3, bill.getSubtotal());
    ps.setDouble(4, bill.getDiscount());
    ps.setDouble(5, bill.getTax());
    ps.setDouble(6, bill.getTotal());
    ps.executeUpdate();
    try (ResultSet rs = ps.getGeneratedKeys()) {
        if (rs.next()) {
            return rs.getInt(1);
        } else {
            throw new SQLException("Failed to retrieve
bill_id.");
        }
    }
}
}
}
}

class PaymentDAO {
    public int savePayment(Payment payment, int dbBillId)
throws SQLException {
        String sql = "INSERT INTO Payments (bill_id,
payment_method, transaction_id, transaction_time, status,
total_paid, total_refunded) " +
            "VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = DBConnection.getConnection());

```

```

        PreparedStatement ps =
conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
    double totalPaid = payment.getTotalPaid();
    double totalRefunded =
payment.getTotalRefunded();
    Payment.Transaction firstTxn = null;
    if (!payment.getTransactions().isEmpty()) {
        firstTxn = payment.getTransactions().get(0);
    }
    if (firstTxn != null) {
        ps.setInt(1, dbBillId);
        ps.setString(2, firstTxn.getMethod().name());
        ps.setString(3, firstTxn.getTransactionId());
        ps.setTimestamp(4,
Timestamp.valueOf(firstTxn.getTime()));
        ps.setString(5, payment.getStatus().name());
        ps.setDouble(6, totalPaid);
        ps.setDouble(7, totalRefunded);
    } else {
        ps.setInt(1, dbBillId);
        ps.setNull(2, Types.VARCHAR);
        ps.setNull(3, Types.VARCHAR);
        ps.setTimestamp(4,
Timestamp.valueOf(LocalDate.now()));
        ps.setString(5, payment.getStatus().name());
        ps.setDouble(6, totalPaid);
        ps.setDouble(7, totalRefunded);
    }
    ps.executeUpdate();

```

```

        try (ResultSet rs = ps.getGeneratedKeys()) {
            if (rs.next()) {
                return rs.getInt(1);
            } else {
                throw new SQLException("Failed to retrieve
payment_id.");
            }
        }
    }
}

// ===== ADD THIS NEW CLASS =====
class CustomerDAO {
    public static void saveCustomer(Customer customer)
throws SQLException {
        String sql = "INSERT INTO customers (customer_id,
name, phone, email) VALUES (?, ?, ?, ?) " +
            "ON DUPLICATE KEY UPDATE name=?,
phone=?, email=?";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps =
conn.prepareStatement(sql)) {
            ps.setInt(1, customer.customerId);
            ps.setString(2, customer.name);
            ps.setString(3, customer.phone);
            ps.setString(4, customer.email);
            ps.setString(5, customer.name);
            ps.setString(6, customer.phone);
            ps.setString(7, customer.email);
            ps.executeUpdate();
        }
    }
}

```



```

    }
}

    public static Customer getCustomer(int customerId)
throws SQLException {
    String sql = "SELECT * FROM customers WHERE
customer_id = ?";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps =
conn.prepareStatement(sql)) {
        ps.setInt(1, customerId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return new Customer(
                    rs.getInt("customer_id"),
                    rs.getString("name"),
                    rs.getString("phone"),
                    rs.getString("email")
                );
            }
        }
    }
    return null;
}

// ===== ADD THIS NEW CLASS =====
class RestaurantTableDAO {
    public static void initializeTables() throws SQLException
{
        String checkSql = "SELECT COUNT(*) FROM
restaurant_tables";

```

```
String insertSql = "INSERT INTO restaurant_tables  
(table_id, capacity, is_available) VALUES (?, ?, ?)";
```

```
try (Connection conn = DBConnection.getConnection();  
     Statement stmt = conn.createStatement();  
     ResultSet rs = stmt.executeQuery(checkSql)) {  
  
    rs.next();  
    int count = rs.getInt(1);  
  
    if (count == 0) {  
        try (PreparedStatement ps =  
conn.prepareStatement(insertSql)) {  
            int tableId = 1;  
            for (int i = 0; i < 8; i++) {  
                ps.setInt(1, tableId++);  
                ps.setInt(2, 4);  
                ps.setBoolean(3, true);  
                ps.addBatch();  
            }  
            for (int i = 0; i < 6; i++) {  
                ps.setInt(1, tableId++);  
                ps.setInt(2, 6);  
                ps.setBoolean(3, true);  
                ps.addBatch();  
            }  
            for (int i = 0; i < 2; i++) {  
                ps.setInt(1, tableId++);  
                ps.setInt(2, 10);  
                ps.setBoolean(3, true);
```

```

        ps.addBatch();
    }
    for (int i = 0; i < 6; i++) {
        ps.setInt(1, tableId++);
        ps.setInt(2, 2);
        ps.setBoolean(3, true);
        ps.addBatch();
    }
    ps.executeBatch();
}
}
}
}

    public static void updateTableStatus(int tableId,
boolean isAvailable) throws SQLException {
        String sql = "UPDATE restaurant_tables SET
is_available = ? WHERE table_id = ?";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement ps =
conn.prepareStatement(sql)) {
            ps.setBoolean(1, isAvailable);
            ps.setInt(2, tableId);
            ps.executeUpdate();
        }
    }

    public static ArrayList<RestaurantTable> loadTables()
throws SQLException {
        ArrayList<RestaurantTable> tables = new ArrayList<>();
        String sql = "SELECT * FROM restaurant_tables
ORDER BY table_id";

```

```

    try (Connection conn = DBConnection.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        while (rs.next()) {
            int id = rs.getInt("table_id");
            int capacity = rs.getInt("capacity");
            boolean isAvailable =
rs.getBoolean("is_available");

            RestaurantTable table = new RestaurantTable(id,
capacity);

            if (!isAvailable) {
                table.occupyTable();
            }
            tables.add(table);
        }
    }
    return tables;
}

// ===== ADD THIS NEW CLASS =====
class ReservationDAO {
    public static int saveReservation(Reservation
reservation) throws SQLException {
        Connection conn = null;
        PreparedStatement ps1 = null, ps2 = null;
        ResultSet rs = null;

```

```

try {
    conn = DBConnection.getConnection();
    conn.setAutoCommit(false);

    CustomerDAO.saveCustomer(reservation.customer);

    String sql1 = "INSERT INTO reservations
(customer_id, date_time, status) VALUES (?, ?, 'ACTIVE')";
    ps1 = conn.prepareStatement(sql1,
Statement.RETURN_GENERATED_KEYS);
    ps1.setInt(1, reservation.customer.customerId);
    ps1.setString(2, reservation.dateTime);
    ps1.executeUpdate();

    rs = ps1.getGeneratedKeys();
    int reservationId = -1;
    if (rs.next()) {
        reservationId = rs.getInt(1);
    }

    String sql2 = "INSERT INTO reservation_tables
(reservation_id, table_id) VALUES (?, ?)";
    ps2 = conn.prepareStatement(sql2);
    for (RestaurantTable table :
reservation.allocatedTables) {
        ps2.setInt(1, reservationId);
        ps2.setInt(2, table.table_id);
        ps2.addBatch();
    }
}

```

```

RestaurantTableDAO.updateTableStatus(table.table_id, false);
    }
    ps2.executeBatch();

    conn.commit();
    return reservationId;

} catch (SQLException e) {
    if (conn != null) {
        try {
            conn.rollback();
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
    throw e;
} finally {
    if (rs != null) rs.close();
    if (ps1 != null) ps1.close();
    if (ps2 != null) ps2.close();
    if (conn != null) {
        conn.setAutoCommit(true);
        conn.close();
    }
}
}

public static ArrayList<Reservation> loadReservations()
throws SQLException {

```

```

        ArrayList<Reservation> reservations = new
ArrayList<>();
        String sql = "SELECT r.reservation_id, r.customer_id,
r.date_time, " +
            "c.name, c.phone, c.email " +
            "FROM reservations r " +
            "JOIN customers c ON r.customer_id =
c.customer_id " +
            "WHERE r.status = 'ACTIVE' " +
            "ORDER BY r.reservation_id";

        try (Connection conn = DBConnection.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                int resId = rs.getInt("reservation_id");
                Customer customer = new Customer(
                    rs.getInt("customer_id"),
                    rs.getString("name"),
                    rs.getString("phone"),
                    rs.getString("email")
                );
                String dateTime = rs.getString("date_time");

                ArrayList<RestaurantTable> allocatedTables =
loadReservationTables(resId);

                Reservation reservation = new Reservation(resId,
customer, allocatedTables, dateTime);

```

```

        reservations.add(reservation);
    }
}
return reservations;
}

private static ArrayList<RestaurantTable>
loadReservationTables(int reservationId) throws SQLException {
    ArrayList<RestaurantTable> tables = new ArrayList<>();
    String sql = "SELECT rt.table_id, rt.capacity,
rt.is_available " +
        "FROM reservation_tables res_t " +
        "JOIN restaurant_tables rt ON res_t.table_id =
rt.table_id " +
        "WHERE res_t.reservation_id = ?";

    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps =
conn.prepareStatement(sql)) {
        ps.setInt(1, reservationId);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                RestaurantTable table = new RestaurantTable(
                    rs.getInt("table_id"),
                    rs.getInt("capacity")
                );
                if (!rs.getBoolean("is_available")) {
                    table.occupyTable();
                }
                tables.add(table);
            }
        }
    }
}

```



```

    }
}
return tables;
}

public static void cancelReservation(int reservationId)
throws SQLException {
    Connection conn = null;
    try {
        conn = DBConnection.getConnection();
        conn.setAutoCommit(false);

        String sql1 = "SELECT table_id FROM
reservation_tables WHERE reservation_id = ?";
        ArrayList<Integer> tableIds = new ArrayList<>();
        try (PreparedStatement ps =
conn.prepareStatement(sql1)) {
            ps.setInt(1, reservationId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    tableIds.add(rs.getInt("table_id"));
                }
            }
        }

        for (int tableId : tableIds) {
            RestaurantTableDAO.updateTableStatus(tableId,
true);
        }
    }
}

```

```

        String sql2 = "UPDATE reservations SET status =
        'CANCELLED' WHERE reservation_id = ?";
        try (PreparedStatement ps =
        conn.prepareStatement(sql2)) {
            ps.setInt(1, reservationId);
            ps.executeUpdate();
        }

        conn.commit();
    } catch (SQLException e) {
        if (conn != null) {
            try {
                conn.rollback();
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
        }
        throw e;
    } finally {
        if (conn != null) {
            conn.setAutoCommit(true);
            conn.close();
        }
    }
}

```

```

// ===== DEMO / USAGE EXAMPLE
=====

```

```

// FIXED: Removed 'public' modifier to avoid conflict with
public class Main

```

```

class RestaurantAppDemo {
    public static void runDemo() {
        Order order = new Order(5, "John Doe", "Birthday");
        order.addItem(new OrderItem("Margherita Pizza",
250.00, 2));
        order.addItem(new OrderItem("Red Sauce Pasta",
350.00, 1));
        order.setDiscount(0);
        order.setTax(5);
        OrderDAO orderDAO = new OrderDAO();
        BillDAO billDAO = new BillDAO();
        PaymentDAO paymentDAO = new PaymentDAO();
        try {
            int dbOrderId = orderDAO.saveOrder(order);
            System.out.println("Saved Order in DB with order_id
= " + dbOrderId);
            Bill bill = new Bill(order);
            int dbBillId = billDAO.saveBill(bill, dbOrderId);
            System.out.println("Saved Bill in DB with bill_id = " +
dbBillId);
            Payment payment = new Payment(bill);
            payment.makePayment(PaymentMethod.UPI,
bill.getTotal());
            int dbPaymentId =
paymentDAO.savePayment(payment, dbBillId);
            System.out.println("Saved Payment in DB with
payment_id = " + dbPaymentId);
            System.out.println("Demo flow completed
successfully.");
        } catch (SQLException ex) {

```

```
        System.err.println("Database error: " +  
ex.getMessage());  
        ex.printStackTrace();  
    } catch (Exception ex) {  
        System.err.println("Runtime error: " +  
ex.getMessage());  
        ex.printStackTrace();  
    }  
}  
}
```